“Theater Ticketing System”

Software Requirements Specification

Version 1.0

August 4, 2026

Group 10
Trinh Ho
William Y Tong
Amina Bendjennat

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Summer 2026

Revision History

Date	Description	Author	Comments
07/09/2026	Version 0.1	Trinh Ho William Y. Tong Amina Bendjennat	Initial Draft
07/19/2026	Version 0.5	Trinh Ho William Y. Tong Amina Bendjennat	Revised draft
07/27/2026	Version 0.7	Trinh Ho William Y. Tong Amina Bendjennat	Revised draft
08/04/2026	Version 1.0	Trinh Ho William Y. Tong Amina Bendjennat	Final Review

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to properly specify the system requirements for a **Theater Ticketing System** before the process of development begins. It serves as an official agreement between the development team and stakeholders by defining the system's functions, constraints, and its expected levels of performance in advance. Features including both online and kiosk ticket purchasing, an account system, payment processing, reservations, security functions, and administrator permissions are included in the requirements.

The intended audience for this SRS includes the following:

- Clients and stakeholders (i.e. theater management) to confirm that the established requirements hold true to their business expectations. Interview responses describe their preferences in terms of features such as funding/budget, project deadlines, and operational limitations.
- Software development and architecture teams using the SRS as their main guide for implementation of the ticketing system depending on requirements.
- Quality assurance testers who utilize the requirements to produce test cases that verify the system fulfills all requirements including proper handling of users and ensuring stability of security measures.
- Project managers utilizing the SRS as a blueprint to estimate the scope and features of their project regarding scheduling and budgeting requirements. The client defines a four-month timeframe for the prototype to be completed within a budget of $500,000.
- Future maintenance teams who reference the SRS when troubleshooting or upgrading the future system after it is launched.

1.2 Scope

The Theater Ticketing System is a web-based system supporting online and in-person theater kiosk ticket sales for a theater chain with locations within the San Diego area. The application allows customers to search through various films and showtimes, place seating reservations, purchase up to 20 tickets per individual transaction, process payments safely and securely, and receive their tickets either printed or digitally. Customers are granted the ability to utilize the system in English, Spanish, or Swedish to create accounts where they may access

loyalty memberships, view movie ratings with critic reviews, and submit personal feedback. Discounts are supported for students, military causes, and senior citizens. Tickets for a specific film may be purchased between two weeks before the viewing and up to ten minutes after the movie has started, allowing late-arriving customers to purchase tickets for admission.

For administrators of the theater system, the application provides proper tools to manage showtimes, seat availability, ticket pricing, monitoring of sales, and handle customer feedback and issues. A main database is used to maintain ticket availability throughout all actively participating theaters in real time.

The system ensures secure processing of customer payments and fully accurate, synchronized seat reservations across purchasing platforms. To enhance ticket authenticity and protect against ticket duplication, each digital ticket shall be issued as an NFT-based digital ticket. The system also incorporates measures to protect against ticket scalping and automated bot purchases.

The initial release of the system excludes support for nearby theater recommendations and customer-initiated refunds. Any refunds directly require administrator approval before action is taken and will follow pre-established theater policies.

1.3 Definitions, Acronyms, and Abbreviations

Term	***Definition***
SRS	Software Requirements Specification, a document that describes the functional and non-functional requirements of the software system.
UI	User Interface, the graphical interface through which users interact with the system.
DBMS	Database Management System, software that stores, manages, and retrieves system data.
Administrator	An authorized user who manages theaters, showtimes, pricing, and customer issues.
Customer	A user who purchases or reserves movie tickets through the website or theater kiosk.
User Account	A registered customer profile containing personal information, login credentials, and booking history.
Registration	The process of creating a new user account in the system.
Loyalty Membership	A customer rewards program that provides members with benefits such as waived online service fees, and waived seat reservation fees.
Showtime	The scheduled date and time a movie is shown.

Transaction	A completed purchase of one or more movie tickets.
Refund	The return of payment to a customer for a canceled transaction according to theater policy.
Kiosk	A self-service terminal located in a theater for purchasing or printing tickets.
Scalping	The practice of purchasing tickets in bulk for resale at higher prices.
Bot	An automated program that attempts to purchase tickets without human interaction.
Concurrent User	A user accessing the system simultaneously with other users.
Database	Collection of all the information monitored by this system.
Stakeholder	Any person, group, or organization that has an interest in the Theater Ticketing System, such as theater management, customers, administrators, developers, testers, and project managers.
Seat Availability	The current status indicating whether a seat is available, reserved, or sold.
Audit Log	A chronological record of significant system activities used for monitoring and security purposes.
Guest	A user who accesses the Theater Ticketing System without creating an account or logging in.
NFT-Based Ticket	A digital movie ticket represented as a Non-Fungible Token (NFT) on a blockchain, providing a unique, verifiable, and tamper-resistant proof of ownership. NFT-based tickets may support secure transfers, fraud prevention, and collectible features.
Member Identification Number (Member ID)	A unique identifier assigned to each loyalty program member for identification, account management, and reward tracking within the Theater Ticketing System.

1.4 References

Institute of Electrical and Electronics Engineers. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998, IEEE, 1998.

1.5 Overview

The remainder of this document is organized into four main sections: General Description, Specific Requirements, Software Design, Analysis Models, Project Management, Links to GitHub and Change Management Process.

The General Description section provides an overview of the Theater Ticketing System by describing the product perspective, product functions, user characteristics, general constraints, and assumptions and dependencies. This section establishes a context for the detailed technical and functional requirements specifications for the next section.

The Specific Requirements section is primarily intended for developers by providing a detailed description of the technical requirements for the Theater Ticketing System and the detailed functionality of the product. It describes the external interface requirements, functional requirements, use cases, non-functional requirements, inverse requirements, design constraints, logical database requirements, and other system requirements.

The Software Design section translates the requirements into a technical design for implementation. It presents the software architecture, the software Architecture diagram, the UML class diagram, and the detailed descriptions of the system's classes, attributes, functions, and relationships.

The Analysis Models section provides graphical representations of the Theater Ticketing System to show the system's behavior and interactions. It includes sequence diagrams of important system processes. This helps both stakeholders and developers understand how users, system components, external services, and the database interact in the system.

The Project Management section provides an overview of the proposed software system, project tasks, estimated project timeline, and responsibilities assigned to each team member.

The Links to GitHub section provides the repository links used by each team member to store project files and contributions.

The Change Management Process section describes the procedures used to make changes to the requirements of the Theater Ticketing System throughout the development lifecycle.

All sections of the document describe the same software product but are intended for different audiences and applications. Therefore, the language and level of detail used in each section may vary depending on its purpose.

2. General Description

This section describes the system from a high-level perspective, including its major functions, intended users, operational constraints, and assumptions. The purpose of this section is to provide context for the detailed functional and non-functional requirements presented in Section 3. It does not define specific system requirements but instead describes the characteristics of the software and the factors that influence its design and implementation.

2.1 Product Perspective

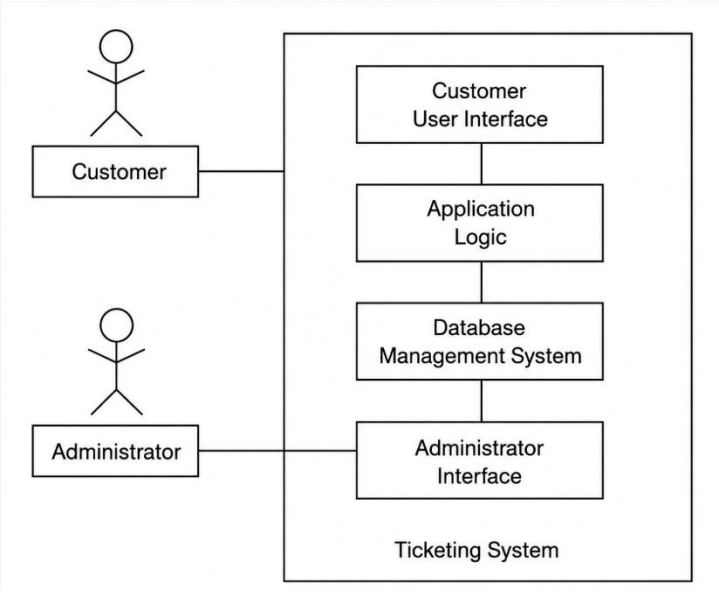


Figure 1 – System Environment

The Theater Ticketing System has two primary actors with several cooperating systems. The customer and administrator both directly access the ticketing system through a web browser. The customer is presented with the Customer User Interface to browse movies, select showtimes, purchase tickets, and manage account information. The administrator accesses the Administrator Interface to manage movies, showtimes, ticket pricing, theater information, and customer transactions. The system processes requests through the application logic, stores information in the Database Management System, and communicates with the payment gateway, email service when required.

2.2 Product Functions

The Theater Ticketing System provides a centralized platform for customers and theater administrators to manage movie ticket sales and theater operations. Customers can browse available movies and showtimes, view seat availability, purchase tickets through the website or in theater kiosks, create and manage customer accounts, and receive printed tickets or NFT-based digital tickets after successful purchases. The system also supports membership benefits, discount eligibility, secure payment processing, and customer feedback.

For administrators, the system provides a secure interface for managing movies, showtimes, theater information, ticket pricing, discounts, seat availability, and customer transactions. The system maintains a centralized database to synchronize ticket inventory across all theaters, communicates with external services such as the payment gateway and email service, and records transaction and administrative information to support efficient theater operations.

2.3 User Characteristics

The Theater Ticketing System is intended for two primary user groups:

- **Customers:** Individuals purchasing movie tickets online or through theater kiosks. Customers are expected to have basic computer and Internet skills, including the ability to use a web browser, search for movies, complete online forms, and make electronic payments. No specialized technical knowledge is required.
- **Administrators:** Authorized theater employees responsible for managing movies, showtimes, ticket pricing, refunds, and customer transactions. Administrators should have basic computer proficiency and receive training on the administrative functions of the system.

2.4 General Constraints

The Theater Ticketing System shall be hosted on a secure server with a reliable, high-speed Internet connection to support continuous operation and real-time ticket processing. The hosting environment will be determined by the theater department and must comply with the organization's existing IT infrastructure and security policies.

The system shall use a web server and a Database Management System (DBMS) to support communication between the website, theater kiosks, and the centralized ticketing database. Administrative functions shall be accessible only to authorized theater staff using secure staff computers with appropriate user permissions.

The performance of the Theater Ticketing System depends on the availability of the Internet connection and the underlying hardware and network infrastructure. Customers are expected to access the system through a supported web browser with a stable Internet connection.

2.5 Assumptions and Dependencies

Assumptions

The following are assumed:

1. Customers can access a reliable internet connection with a supported web browser while utilizing the application.
2. A continuous connection is present between theater kiosks and the main server throughout official operating hours.
3. Administrators are effectively trained to use the administrator interface in advance.
4. The personal information used for registration and purchases is properly entered and accurate.
5. Valid necessary documentation and identification information is presented by customers using the student, senior, and military discount features.

Dependencies

The system is dependent on the following:

1. A certified emailing service is utilized for direct interactions regarding account verification, confirmations, notifications, advertising, and password recovery.
2. A secure third-party gateway is used to process payments and refunds from customers.
3. The main Database Management System stores all information regarding customer accounts, ticket information, film schedules, seating availability, and transaction information.
4. External services specializing in film information are used to communicate film ratings and reviews from expert movie critics.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Theater Ticketing System shall provide separate user interfaces for customers and administrators.

- Customers shall access the system through a responsive web interface using a supported web browser.
- The customer interface shall allow users to browse movies, search showtimes, select seats, purchase tickets, manage customer accounts, and submit feedback.

- Administrators shall access the system through a secure administrative interface after successful authentication.
- The system shall support the following display languages: English, Spanish, and Swedish.
- The administrative interface shall allow authorized staff to manage movies, showtimes, ticket pricing, refunds, customer transactions, and reports.
- Theater kiosks shall provide a touch-screen interface for purchasing and printing tickets.

3.1.2 Hardware Interfaces

The Theater Ticketing System shall interact with the following hardware components:

- Customer desktop computers, laptops, tablets, and smartphones.
- Theater self-service kiosks equipped with touch-screen displays.
- Ticket printers connected to theater kiosks.
- Secure servers hosting the application and database.
- Staff computers used by authorized administrators.

3.1.3 Software Interfaces

The Theater Ticketing System shall communicate with the following software components:

- A Database Management System (DBMS) for storing customer accounts, movies, showtimes, transactions, and feedback.
- An authorized third-party payment gateway for processing customer payments and refunds.
- An email service for sending account verification, ticket confirmations, and refund notifications.
- Movie information services, such as IMDb and Rotten Tomatoes, to retrieve ratings and critic reviews.

3.1.4 Communications Interfaces

The Theater Ticketing System shall communicate over secure Internet connections.

- All communication between users and the system shall use HTTPS.
- Communication with the payment gateway and email service shall use secure APIs.
- Data transmitted between the web application, kiosks, and the database shall be encrypted using TLS.
- The system shall support standard TCP/IP networking protocols.

3.2 Functional Requirements

3.2.1 Ticket Purchase

3.2.1.1 Introduction

The system shall allow customers to purchase movie tickets through the website or in-theater kiosks.

3.2.1.2 Inputs

1. The system shall accept the following inputs from customers:
    a. Customer Selection: Movie title, showtime (theater date and time), and seat

b. Ticket Quantity: 1-20 tickets per transaction
c. Ticket type: regular, student, military, senior
d. Payment method: Visa, MasterCard, PayPal, American Express
e. Billing details

3.2.1.3 Processing

1. The system shall show customers all available movies and showtimes across all 20 San Diego theater locations:
    a. Movie title, runtime, IMDb rating
    b. Theater location
    c. Showtime
    d. Seat availability and seat map
2. The system shall display ticket availability within a purchase window starting from 2 weeks prior to showtime and ending 10 minutes after showtime starts.
3. The system shall limit the maximum number of tickets per single transaction to 20 tickets.
4. The system shall reserve the selected seats for 5 minutes upon seat selection during the purchasing process.
5. The system shall calculate ticket prices based on the base ticket price, local taxes, fees, and local currency conversion if needed.
6. The system shall apply a 5% online service fee and a $2.99 seat reservation fee to guest customer ticket purchases.
7. The system shall apply eligible discounts based on customer categories: student, military, and senior.
8. The system shall accept Visa, MasterCard, American Express, and PayPal as a valid payment method for ticket purchases.
9. The system shall maintain consistent ticket pricing across the website and in-theater kiosks for all 20 theater locations.
10. The system shall generate unique, non-replicable NFT-based ticket identifiers for each ticket after a successful payment.
11. The system shall request an email address for ticket delivery when a customer purchases a ticket on the website without logging into a customer account.
12. The system shall update seat availability information after successful ticket purchase.
13. The system shall create and store a transaction record in the database after a successful ticket purchase.

3.2.1.4 Outputs

1. The system shall display ticket confirmation including showtime details, seat numbers, ticket prices, and total amount charged.
2. The system shall send a digital ticket to the registered email address of a logged-in customer after a successful online ticket purchase.
3. The system shall send a digital ticket to the email address provided during checkout by a guest customer after a successful online ticket purchase.
4. The system shall provide a printed ticket after a successful ticket purchase at in-theater kiosk.
5. The system shall store the completed order transaction in the customer’s account purchase history.

3.2.1.5 Error Handling

1. The system shall cancel the transaction if payment processing fails.
2. The system shall display a clear message and reject the transaction if a customer attempts to buy more than 20 tickets.
3. The system shall automatically abandon incomplete transactions after the 5-minute seat reservation period and display an error message to the customer.
4. The system shall prevent the payment processing and display an error message if a database connection failure occurs during checkout.

3.2.2 Administrator Management

3.2.2.1 Introduction

The system shall have a capability to manually correct customer errors, manage movies, showtimes, theater locations, ticket pricing, discounts, and refunds through a secure administrative interface for authorized theater administrators.

3.2.2.2 Input

1. The system shall accept the following inputs from administrators:
    a. Administrator login information: Username and password -
    b. Movie information: title, description, runtime, rating, genre, release information
    c. Showtime Information: Theater location, show date, and showtime
    d. Ticket Pricing Information: Ticket prices and pricing adjustments
    e. Refund Information: transaction details, refund amounts, and refund reasons.
    f. Transaction Corrections: Transaction IDs and correction details for purchase errors

3.2.2.3 Processing

1. The system shall authenticate the administrator using the provided username and password.
2. The system shall require the administrator to complete multi-factor authentication before granting access to the administrative interface.
3. The system shall allow authenticated administrators to log out of the administrative interface.
4. The system shall provide the following override capabilities for **authorized administrators**:
    - Transaction Correction
        a. The system shall cancel completed transactions and reverse the charges only for tickets purchased using a customer account when the refund is processed before the scheduled showtime.
        b. The system shall update the transaction records to reflect cancellations, refunds, and any corrections.
        c. The system shall reissue tickets when a customer loses a ticket.
    - Showtime Management
        a. The system shall create new showtimes for new movies at any theater
        b. The system shall modify the existing showtime if necessary
        c. The system shall modify release information for both existing and future movies.
        d. The system shall remove the existing showtime.

- Seat Management
    a. The system shall adjust seat availability for any showing.
    b. The system shall not override a seat already sold to customers.
    c. The system shall block specific seats from being sold.
- Discounts
    a. The system shall update ticket prices
    b. The system shall apply, modify, or remove discounts for students, military, and senior ticket categories.
- Reissue Tickets
    a. The system shall allow an authorized administrator to reissue a physical copy of an existing valid ticket after verifying the original transaction.

3.2.2.4 Outputs

1. The system shall display a confirmation message after a successful administrative operation.
2. The system shall update ticket inventory and seat availability based on administrator changes.
3. The system shall log each administrative operation with the timestamp, and administrator's identity.
4. The system shall notify customers when an administrator's action affects their existing reservations and transactions.
5. The system shall update the ticket status in the customer's purchase history to "Refunded" after a refund has been processed by an authorized theater employee.
6. The system shall invalidate the generated NFT-based digital ticket after a refund has been processed by an authorized theater employee.

3.2.2.5 Error Handling

1. The system shall deny access to users who fail administrator authentication or multi-factor authentication.
2. The system shall reject attempts to create overlapping showtimes.
3. The system shall display an error message if the requested showtime, or movie record does not exist.
4. The system shall display a clear error message when an administrator attempts to process a refund for a ticket purchased by a guest customer.
5. The system shall reject attempts to override an existing ticket reservation by assigning a sold seat to another customer.
6. The system shall prevent any incomplete administrative changes when a system or database error occurs.

3.2.3 Customer Account Management

3.2.3.1 Introduction

The system shall allow customers to create customer accounts that automatically enroll them in the theater's membership program. The system shall allow customers to update their personal information, preferred payment methods, two-factor authentication settings, view purchase history, access purchased ticket information and manage their membership information and benefits on the website.

3.2.3.2 Input

1. The system shall accept the following inputs from customers:
    a. Customer registration information: username, email address, and password.
    b. Payment method registration: Visa, MasterCard, American Express, or PayPal
    c. Two-Factor authentication code (if enabled)

3.2.3.3 Processing

1. The system shall allow customers to register a customer account using a valid email address and password.
2. The system shall authenticate customers using their registered email address and password.
3. The system shall authenticate customers using two-factor authentication when two-factor authentication is enabled.
4. The system shall generate a unique membership identification number for each customer after a successful account registration.
5. The system shall allow customers to update their personal information and preferred payment methods.
6. The system shall allow customers to enable or disable two-factor authentication for their customer accounts.
7. The system shall store the customer's purchase history for all completed transactions made by using the customer account.
8. The system shall waive online service fees and seat reservation fees for customers who purchase tickets while logged into their customer accounts.
9. The system shall restrict each customer account to one active login session on a single device at a time.
10. The system shall allow customers to purchase tickets using the same ticket purchasing process available to guest customers.
11. The system shall allow customers to log out of their accounts.

3.2.3.4 Outputs

1. The system shall send an email confirmation after a successful customer account registration.
2. The system shall display the customer's personal information, preferred payment methods, membership information, and membership identification number.
3. The system shall display the customer's purchase history for all completed transactions made using the customer account.
4. The system shall display a confirmation message after successfully updating the customer's two factor authentication settings.
5. The system shall display the waived online service fee and seat reservation fee during checkout for customers logged into their customer accounts.
6. The system shall send a confirmation email containing the generated NFT-based digital ticket after a successful ticket purchase using a customer account.
7. The system shall display the generated NFT-based digital ticket in the customer's purchase history after a successful ticket purchase using a customer account.

3.2.3.5 Error handling

1. The system shall display an error message when a customer attempts to register using an invalid email address.

2. The system shall display a generic error message without indicating which credential is incorrect when a customer attempts to log in with an invalid email address or password.
3. The system shall display a clear error message when a customer enters an invalid two-factor authentication code.
4. The system shall temporarily prevent login after multiple unsuccessful login attempts.
5. The system shall display an error message when the customer account information cannot be saved or updated.
6. The system shall display an error message when a customer attempts to access the account that is already active on another device.
7. The system shall display an error message when two-factor authentication settings cannot be updated.

3.2.4 Search Movies

3.2.4.1 Introduction

The system shall allow customers to search for available movies by movie title, actor name, actress name, genre, showtime, and theater location through the website or in-theater kiosks.

3.2.4.2 Inputs

1. The system shall accept the following input:
    a. Movie title
    b. Actor Name
    c. Actress Name
    d. Movie genre
    e. Showtime (optional)
    f. Theater location (optional)

3.2.4.3 Processing

1. The system shall search for available movies based on the customer's entered keywords (movie title, actor name, actress name, genre, showtime, or theater location)
2. The system shall retrieve matching movie records from the centralized database.
3. The system shall retrieve available showtimes, and theater locations for matching movies
4. The system shall display matching movies, available showtimes, and available theater locations to the customer.
5. The system shall sort results according to the search criteria.

3.2.4.4 Outputs

1. The system shall display all movies matching the customer's search criteria.
2. The system shall display the movie title, runtime, rating, genre, theater location, and available showtimes for each matching movie.
3. The system shall display Rotten Tomatoes and IMDb ratings together with critic quotes for the selected movie.
4. The system shall allow customers to select the desired movie, theater location, and showtime.
5. The system shall display seat availability and the seat map for the selected showtime.
6. The system shall allow customers to select available seats and continue to the ticket purchase process.

3.2.4.5 Error Handling

1. The system shall display a message when no movies match the customer's search criteria.
2. The system shall display a clear error message if the search cannot be completed because of the database error.
3. The system shall display an appropriate message when movie ratings or critic quotes are unavailable.
4. The system shall prompt the customer to enter at least one search keyword if the search is submitted without any input.

3.2.5 Customer Feedback

3.2.5.1 Introduction

The system shall allow customers to optionally submit feedback after a successful ticket purchase.

3.2.5.2 Inputs

1. The system shall accept the following inputs:
    a. Customer satisfaction rating (smiley face rating scale)
    b. Comments

3.2.5.3 Processing

1. The system shall display the customer feedback form after a successful ticket purchase.
2. The system shall display a range of smiley face icons representing a customer satisfaction level from very satisfied to very dissatisfied.
3. The system shall allow customers to select one smiley-face rating.
4. The system shall allow customers to enter optional comments.
5. The system shall store customers' feedback associated with the completed ticket transaction in the centralized ticketing database.

3.2.5.4 Outputs

1. The system shall display a confirmation message after a successful feedback submission.
2. The system shall store the submitted feedback for management review.

3.2.5.5 Error Handling

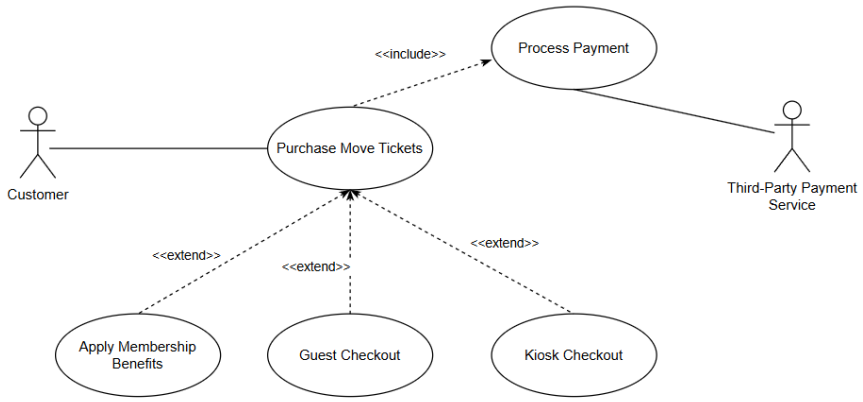
1. The system shall display an error message if the feedback cannot be submitted.
2. The system shall notify the customer that the feedback submission failed and allow the customer to retry the submission.

3.3 Use Cases

3.3.1 Use Case 1: Customer Purchases Movie Ticket

Name of Use Case: Purchase Movie Tickets
Actors: Customer, Third-Party Payment Service

Diagram:



Brief Description:

The customer purchases movie tickets through the theater website or an in-theater kiosk.

Flow of Events (Main):

1. The customer chooses to purchase movie tickets.
2. The system displays all available movies, showtimes, and theater locations, and search options.
3. The customer browses the available movies or searches for a movie.
4. The customer selects the desired movie, showtime, and theater location.
5. The system displays available seats and the seating map.
6. The customer selects the desired seats, and ticket quantity.
7. The customer selects the ticket category (regular, student, military, or senior).
8. The system notifies the customer that valid identification must be presented at the admission for student, military, or senior discounted tickets.
9. The system calculates and displays the total ticket price, including applicable taxes, fees, eligible discounts, and currency conversion (if applicable).
10. The customer selects a payment method and enters the required payment information.
11. The system submits the payment to the authorized third-party payment service.
12. The third-party payment service authorizes the payment.
13. The system generates an NFT-based digital ticket for each purchased ticket.
14. The system updates seat availability in the centralized ticketing database and stores the completed transaction.
15. The system sends a confirmation email containing the NFT-based ticket(s) if the purchase is completed on the website.
16. The system displays the ticket purchase confirmation.

Alternative Flows:

1. Member Purchase
   Occurs before step 9.

I. The system detects that the customer is logged into a customer account.
II. The system waives the online service fee and seat reservation fee.
III. The use case continues at step 9.

2. Guest Purchase
   Occurs before step 9.
   I. The system detects that the customer is not logged into a customer account.
   II. The system requests an email address for ticket delivery.
   III. The customer provides a valid email address
   IV. The use case continues at step 9.
3. Payment failure
   Occurs after step 12.
   I. The third-party payment service notifies the system that payment authorization has failed.
   II. The system displays an error message.
   III. The customer may retry the payment.
   IV. the use case continues at step 10.
4. Ticket Quantity Limit Exceeded
   Occurs after step 6.
   I. The customer selects more than 20 tickets.
   II. The system displays an error message.
   III. The customer modifies the ticket quantity
   IV. The use case continues at step 6.
5. Kiosk Purchase
   Occurs after step 14.
   I. The system prints the physical tickets if the purchase is completed at an in-theater kiosk.
   II. The use case continues at step 16.

Entry Conditions:

1. The theater ticketing system is operational.
2. If the customer wants to receive membership benefits, the customer must be logged into their customer account.
3. The in-theater kiosk is operational if the purchase is made at a kiosk.
4. The ticket purchase occurs within the allowed purchase window (from 2 weeks before the scheduled showtime until 10 minutes after the scheduled showtime starts).

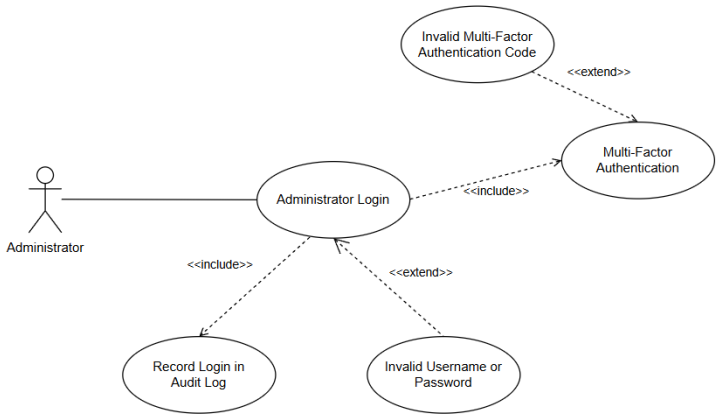
Exit Conditions:

1. Success:
   - Payment is successfully processed
   - NFT-based tickets are generated
   - Seat availability is updated
   - The completed transaction is stored in the centralized ticketing database
   - A confirmation email is sent for website purchases
   - A physical ticket is printed for kiosk purchases
   - The purchase history is updated for logged-in members.
2. Failure:

- The transaction is canceled
- Reserved seats are released
- No payment is processed and no ticket is generated.

3.3.2 Use Case 2: Administrator Login

Name of Use Case: Administrator Login
Actors: Administrator
Diagram:



Brief Description:

The administrator logs into the administrative interface using valid username and password credentials. Multi-factor authentication is required.

Flow of Events (Main):

1. The administrator accesses the administrative login page.
2. The system requests the administrator's username and password.
3. The administrator enters the username and password.
4. The system validates the administrator's login credentials.
5. The system prompts the administrator to enter a multi-factor authentication code.
6. The administrator enters the multi-factor authentication code.
7. The system verifies the multi-factor authentication code.
8. The system grants access to the administrative interface.
9. The system records the administrator's login in the audit log.

Alternative Flows:

1. Invalid Credentials
   Occurs at Step 4.
   I. The system cannot verify the administrator's login credentials.
   II. The system rejects the login attempt.
   III. The system displays a generic error message.
   IV. The use case ends.
2. Invalid Multi-Factor Authentication Code

Occurs at Step 7.

I. The system cannot verify the multi-factor authentication code.
II. The system rejects the login attempt.
III. The administrator may re-enter the authentication code.
IV. The use case continues at Step 5.

Entry Conditions:

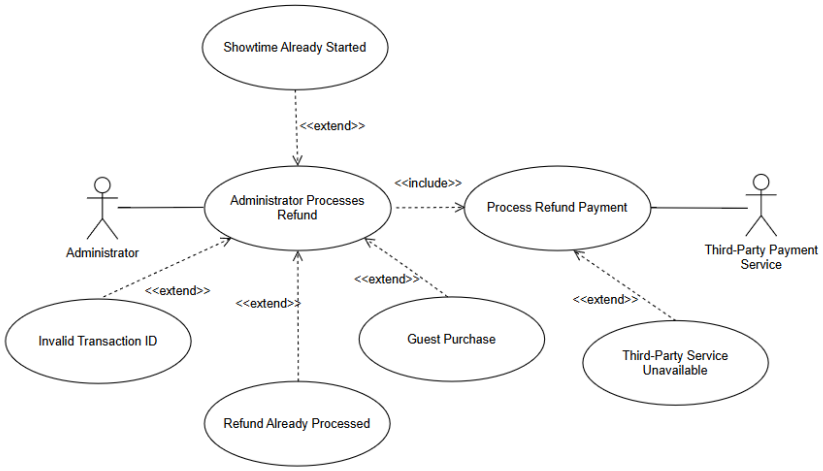
1. The administrator has a valid administrator account.
2. The theater ticketing system is operational.
3. The administrator has access to the registered multi-factor authentication device.

Exit Conditions:

1. Success
   - The administrator is successfully authenticated.
   - The administrator has access to the administrative interface.
   - The successful login is recorded in the audit log.
2. Failure
   - The administrator is denied access to the administrative interface.
   - The unsuccessful login is recorded in the audit log.

3.3.3 Use Case 3: Administrator Process Refund

Name of Use Case: Administrator Processes Refund
Actors: Administrator, Third-Party Payment Service
Diagram:



Brief Description:

The administrator processes a refund for an eligible ticket using the customer's transaction ID.

Flow of Events (Main):

1. The administrator selects the refund processing option.
2. The system requests the administrator to enter the transaction ID.
3. The administrator enters the transaction ID.
4. The system retrieves the transaction record.
5. The system verifies that the transaction was completed through a customer account.
6. The system verifies that the scheduled showtime for the ticket has not started.
7. The system requests the administrator to confirm the refund process.
8. The administrator confirms the refund process.
9. The system submits the refund request to the authorized third-party payment service.
10. The third-party payment service approves the refund.
11. The system updates the ticket status to “Refunded”
12. The system invalidates the NFT-based digital ticket.
13. The system updates seat availability in the centralized ticketing database.
14. The system records the refund transaction in the audit log
15. The system displays a refund confirmation message to the administrator
16. The system sends a refund confirmation email to the customer.

Alternative Flows:

1. Invalid Transaction ID
   Occurs at Step 4.
   I. The system cannot locate the transaction record.
   II. The system displays a clear error message.
   III. The administrator may enter another transaction ID.
   IV. The use case continues at Step 2.
2. Refund Already Processed
   Occurs at Step 4.
   I. The system detects that the ticket has already been refunded.
   II. The system displays a clear error message.
   III. The use case ends.
3. Guest Purchase
   Occurs at Step 5.
   I. The system detects that the ticket purchase was done by a guest user.
   II. The system displays a clear error message indicating that the refund process is only available for tickets purchased through the customer account.
   III. The use case ends.
4. Showtime Already Started
   Occurs at Step 6.
   I. The system detects that the scheduled showtime has already started.
   II. The system displays a clear error message indicating that the refund cannot be processed after the scheduled showtimes has started.
   III. The use case ends.
5. Third-Party Payment Service Unavailable
   Occurs after Step 9.
   I. The third-party Payment Service notifies that the refund could not be processed.
   II. The system displays a clear error message.

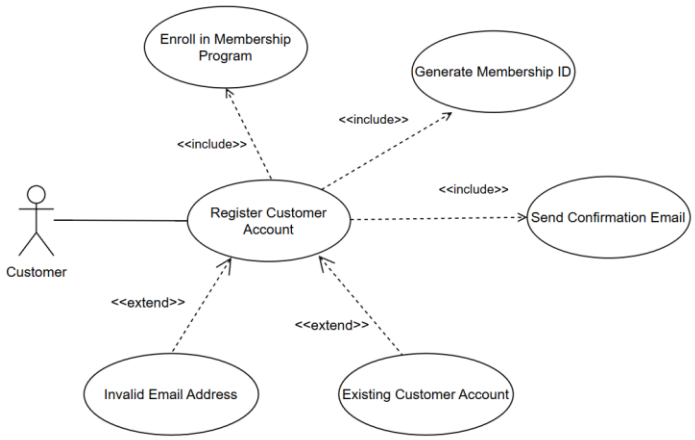
III. The administrator may retry the refund.
IV. The use case continues at Step 9.

Entry Conditions:
1. The theater ticketing system is operational.
2. The administrator has successfully logged into the administrative interface.
3. The customer provides a correct transaction ID.
4. The scheduled showtime has not started.

Exit Conditions:
1. Success
   - The refund is successfully processed
   - The ticket status is updated to "Refunded."
   - The refunded seat becomes available for purchase.
   - The refund transaction is recorded in the audit log.
   - A refund confirmation email is sent to the customer.
2. Failure
   - The refund is not processed.
   - The seat remains unavailable for future purchases.
   - No changes are made to the transaction record.

3.3.4 Use Case 4: Register Customer Account

Name of Use Case: Register customer account
Actors: Customer
Diagram:



Brief Description:
The customer creates an account through the theater website and is automatically enroll in the theater's membership program.

Flow of Events (Main):

1. The customer selects the registration option.
2. The system displays the customer account registration form, including email address, username, and password.
3. The customer enters the required information.
4. The system validates the registration information.
5. The system creates the customer account.
6. The system enrolls the customer in the theater's membership program.
7. The system generates a unique membership identification number.
8. The system sends a confirmation email after successful account registration.
9. The system displays a registration confirmation message.

Alternative Flows:

1. Invalid Email Address
   Occurs at Step 4.
   I. The system detects an invalid email address.
   II. The system displays a clear error message.
   III. The customer may reenter the valid email address.
   IV. The use case continues at Step 4.
2. Existing Customer Account
   Occurs at Step 4.
   I. The system detects that the email address is already associated with an existing account.
   II. The system displays a clear error message.
   III. The use case ends.

Entry Conditions:

1. The theater ticketing system is operational.
2. The customer has access to the internet and the theater website through a supported web browser.
3. The customer does not already have a customer account associated with the registered email address.

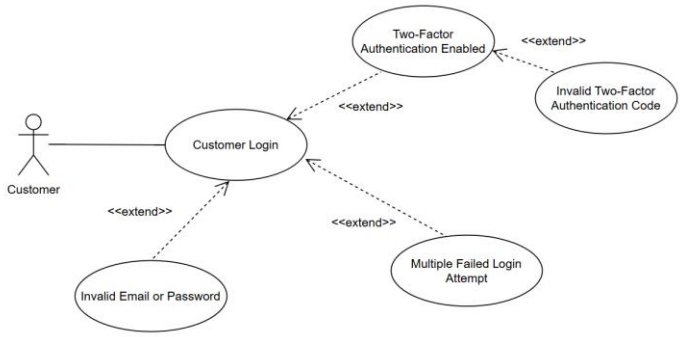
Exit Conditions:

1. Success
   - The customer account is created.
   - The customer is enrolled in the theater's membership program.
   - A unique membership identification number is generated.
   - A confirmation email is sent to the customer.
2. Failure
   - The customer account is not created.

3.3.5 Use Case 5 Customer Login

Name of Use Case: Customer Login
Actors: Customer

Diagram:



Brief Description:

The customer logs into their account using a registered email address and password. Two-factor authentication is optional.

Flow of Events (Main):

1. The customer selects the login option.
2. The system displays the login page.
3. The customer enters the email address and password.
4. The system validates the customer's login credentials.
5. The system grants the customer access to the customer account.
6. The system records the successful login.

Alternative Flows:

1. Two-Factor Authentication Enabled
   Occurs after Step 4.
   I. The system detects that the customer has enabled the two-factor authentication.
   II. The system prompts the customer to enter the two-factor authentication code.
   III. The customer enters the two-factor authentication code.
   IV. The system verifies the two-factor authentication code.
   V. The use case continues at Step 5.
2. Invalid Email Address or Password
   Occurs at Step 4.
   I. The system cannot verify the customer's login credentials.
   II. The system displays a generic message without indicating which credential is incorrect.
   III. The customer may reenter the login credentials.
   IV. The use case continues at Step 3.
3. Invalid Two-Factor Authentication Code
   Occurs at Alternative Flow 1, Step IV.
   I. The system cannot verify the two-factor authentication code.
   II. The system displays a clear error message.

III. The customer may reenter the two-factor authentication code.
IV. The use case continues at Alternative Flow 1. Step III.

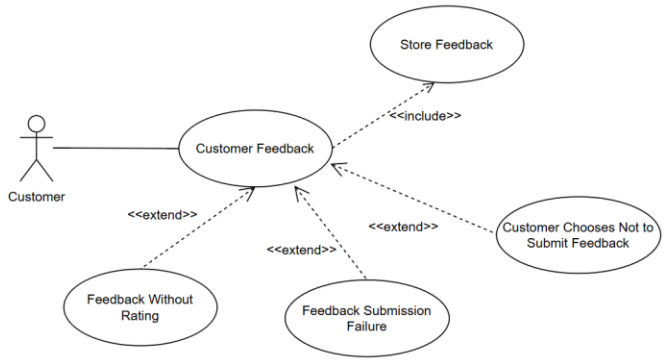
4. Multiple Failed Login Attempts
   Occurs at Step 4.
   I. The system detects that the customer exceeds the maximum number of unsuccessful login attempts.
   II. The system temporarily prevents the customer from logging in.
   III. The system displays an error message.
   IV. The use case ends.

Entry Conditions:
1. The theater ticketing system is operational.
2. The customer has a registered customer account.
3. The customer has access to the internet and the theater website through a supported web browser.
4. The customer is not currently logged into the same account on another device.

Exit Conditions:
1. Success
   - The customer is successfully authenticated.
   - The customer is granted access to the customer account.
   - The successful login event is recorded in the audit log.
2. Failure
   - The customer is denied access to the customer account.
   - The unsuccessful login event is recorded in the audit log.

3.3.6 Use Case 6: Customer Feedback

Name of Use Case: Cusstomer Feedback
Actors: Customer
Diagram:



Brief Description:

The customer submits feedback after a successful ticket purchase by selecting a smiley-face satisfaction rating and providing optional comments.

Flow of Events (Main):

1. The customer completes a ticket purchase.
2. The system displays the customer feedback form.
3. The system displays a range of smiley-face icons that represent customer satisfaction levels from very satisfied to very dissatisfied.
4. The customer selects one smiley-face rating.
5. The customer enters optional comments.
6. The customer submits the feedback.
7. The system stores the submitted feedback with a timestamp in the centralized ticketing database and associates it with the completed ticket transaction.
8. The system displays a confirmation message after a successful feedback submission.

Alternative Flow:

1. Customer Chooses Not to Submit Feedback
   Occurs after Step 2.
   I. The customer closes the feedback form.
   II. The use case ends.
2. Feedback Submission Failure
   Occurs after Step 6.
   I. The customer cannot submit the feedback because of a system error.
   II. The system displays an error message.
   III. The customer may retry the feedback submission.
   IV. The use case continues at Step 6.
3. Feedback Without a Rating
   Occurs after Step 6.
   I. The system detects that the customer submitted the feedback without selecting a smiley-face rating.
   II. The system displays an error message requesting the customer to select a satisfaction rating.
   III. The customer selects a smiley-face rating.
   IV. The use case continues at Step 6.

Entry Conditions:

1. The customer has successfully completed a ticket purchase.
2. The theater ticketing system is fully operational.

Exit Conditions:

1. Success
   - The customer feedback is successfully stored in the centralized ticketing database.
   - The feedback is associated with the completed ticket transaction.
   - The customer receives a confirmation message.
2. Failure
   - The feedback is not submitted.

- No feedback is stored in the centralized ticketing database.

3.4 Classes / Objects

3.4.1 Customer

3.4.1.1 Attributes

- Customer ID

3.4.1.2 Functions

- Purchase Tickets
- Submit Feedback

References:

- Functional Requirement **3.2.1** – Ticket Purchase
- Functional Requirement **3.2.5** – Customer Feedback
- Use Case **3.3.1** – Purchase Movie Tickets
- Use Case **3.3.6** – Customer Feedback

3.4.2 Guest

3.4.2.1 Attributes

- Email Address

3.4.2.2 Functions

- Inherited from Customer Class

References:

- Functional Requirement **3.2.1** – Ticket Purchase
- Functional Requirement **3.2.5** – Customer Feedback
- Use Case **3.3.1** – Purchase Movie Tickets
- Use Case **3.3.6** – Customer Feedback

3.4.3 RegisteredCustomer

3.4.3.1 Attributes

- Name
- Email Address
- Password
- Membership ID
- Purchase History
- Preferred Payment Method
- Two Factor Authentication Status

3.4.3.2 Functions

- Register Account
- Update Personal Information

- View Purchase History
- Enable/Disable Two-Factor Authentication

References:
- Functional Requirement **3.2.3** – Customer Account Management
- Functional Requirement **3.2.5** – Customer Feedback
- Use Case **3.3.4** – Register Customer Account

3.4.4 Administrator

3.4.4.1 Attributes
- Administrator ID
- Username
- Email Address
- Password
- MFA status
- Access Level

3.4.4.2 Functions
- Manage Movies
- Manage Showtimes
- Manage Ticket Pricing
- Process Refunds
- Reissue Tickets
- Adjust Seat Availability
- View Transaction Records

References:
- Functional Requirement **3.2.2** – Administrator Management
- Use Case **3.3.3** – Administrator Processes Refund

3.4.5 Account Authentication

3.4.5.1 Attributes
- None

3.4.5.2 Functions
- Customer Log In
- Administrator Log In
- Log Out
- Verify Multifactor Authentication

References:
- Functional Requirement 3.2.3 – Customer Account Management
- Functional Requirement 3.2.2 – Administrator Management
- Use Case **3.3.5** – Customer Login

- Use Case **3.3.2** – Administrator Login

3.4.6 Movie

3.4.6.1 Attributes

- Movie ID
- Administrator ID
- Title
- Cast Members
- Genre
- Runtime
- IMDb Rating
- Rotten Tomatoes Rating
- Critic Quotes
- Description
- Release Date

3.4.6.2 Functions

- Display Movie Information
- Update Movie Information
- Display Movie Ratings
- Display Critic Reviews

References:

- Functional Requirement **3.2.2** – Administrator Management
- Use Case **3.3.1** – Purchase Movie Tickets

3.4.7 Showtime

3.4.7.1 Attributes

- Showtime ID
- Movie ID
- Theater ID
- Administrator ID
- Date
- Time
- Seat Availability

3.4.7.2 Functions

- Create Showtime
- Update Showtime
- Remove Showtime
- Display Seat Availability
- Reserve Seats
- Update Seat Availability

References

- Functional Requirement **3.2.1** – Ticket Purchase
- Functional Requirement **3.2.2** – Administrator Management
- Functional Requirement **3.2.4** – Search Movies
- Use Case **3.3.1** – Purchase Movie Tickets

3.4.8 MovieCatalog

3.4.8.1 Attributes

- None

3.4.8.2 Functions

- Retrieve Available Movies
- Search Movies

References:

- Functional Requirement **3.2.4** – Search Movies
- Use Case 3.3.1 – Purchase Movie Tickets

3.4.9 Ticket

3.4.9.1 Attributes

- Ticket ID
- Customer ID
- Transaction ID
- Showtime ID
- Administrator ID
- Seat Number
- Ticket Type
- Price
- Ticket Status
- NFT Identifier

3.4.9.2 Functions

- Validate Ticket
- Cancel Ticket
- Invalidate Ticket
- Display Ticket Information

References:

- Functional Requirement **3.2.1** – Ticket Purchase
- Functional Requirement **3.2.2** – Administrator Management
- Functional Requirement **3.2.3** – Customer Account Management
- Use Case **3.3.1** – Purchase Movie Tickets
- Use Case **3.3.3** – Administrator Processes Refund

3.4.10 Transaction

3.4.10.1 Attributes

- Transaction ID
- Customer ID
- Payment Method
- Transaction Date
- Total Amount
- Transaction Status

3.4.10.2 Functions

- Process Payment
- Record Transaction
- Process Refund
- Update Transaction Status

References:

- Functional Requirement **3.2.1** – Ticket Purchase
- Functional Requirement **3.2.2** – Administrator Management
- Functional Requirement **3.2.3** – Customer Account Management
- Use Case **3.3.1** – Purchase Movie Tickets
- Use Case **3.3.3** – Administrator Processes Refund

3.4.11 Feedback

3.4.11.1 Attributes

- Feedback ID
- Customer ID
- Transaction ID
- Rating
- Comments
- Submission Date

3.4.11.2 Functions

- Store Feedback
- View Feedback
- Manage Feedback

References:

- Functional Requirement **3.2.5** – Customer Feedback
- Use Case **3.3.6** – Customer Feedback

3.4.12 Theater

3.4.12.1 Attributes

- Theater ID
- Theater Name

- Location
- Total Screens

3.4.12.2 Functions
- View Showtimes

3.5 Non-Functional Requirements

3.5.1 Performance

3.5.1.1 Concurrent user Performance
1. The system shall support at least of 10 million concurrent users without degradation of system functionality.

3.5.1.2 System Performance
1. The system shall synchronize tickets availability, ticket purchases, and seat reservations between the online website and in-theater kiosks in real time.
2. The system shall implement a queueing mechanism to manage high volumes of concurrent ticket purchase requests for individual showings.
3. The system shall process at least 95% of transactions in less than a second, and at least 99% of transactions in less than 8 seconds.
4. The system shall determine if a customer is a bot within 2 seconds when a customer begins the ticket purchase section.
5. The system shall update ticket availability in the centralized ticketing database within 3 seconds of ticket purchase completion.
6. The system shall create a summary of ticket purchases for all theaters within 10 seconds.

3.5.1.3 Ticket Delivery Performance
1. The system shall send a confirmation email to the customer within 5 minutes after a successful online ticket transaction.
2. The system shall print and provide physical tickets to the customer within 3 seconds after a successful transaction at an in-person kiosk.

3.5.2 Reliability

3.5.2.1 System Reliability
1. The system shall minimize downtime during scheduled updates.
2. The system shall detect when the payment service gateway becomes unavailable within 30 seconds.
3. The system shall handle existing active transactions gracefully and prevent data corruption during database connection failures.
4. The system shall provide essential services during partial system failures.
5. The system shall detect critical system failures and generate automated email alerts to administrators within 10 minutes.

3.5.2.2 Mean Time Between Failures (MTBF)
1. The system shall achieve a Mean Time Between Failures (MTBF) of at least 30 days without requiring administrator intervention.

3.5.2.3 Mean Time To Recovery (MTTR)

1. The system shall automatically recover from non-critical failures within 10 minutes.
2. The system shall restore services within 1 hour after a critical failure requiring manual administrator intervention.

3.5.3 Availability

3.5.3.1 System Availability

1. The system shall remain available 24 hours per day, 7 days per week, including weekends and holidays.
2. The system shall maintain 99.0% uptime during peak periods, major movie releases and major holidays.
3. Planned maintenance and system updates shall be scheduled during the low-demand periods to reduce disruption to users.
4. Planned maintenance and system updates shall not exceed 2.5 hours in duration.
5. The system shall notify administrators at least 72 hours prior to planned maintenance.

3.5.4 Security

3.5.4.1 Ticket Security

1. The system shall assign each ticket a unique NFT-based ticket.
2. The system shall ensure that valid tickets cannot be duplicated or forged.
3. The system shall prevent automated bots from mass purchasing tickets.

3.5.4.2 Customer Account Security

1. The system shall require login credentials for registered customers to access their accounts.
2. The system shall provide 2-Factor Authentication as an optional security feature for customer accounts.
3. The system shall prevent customers from accessing the same account simultaneously from multiple devices

3.5.4.3 Administrator Account Security

1. The system shall require login credentials for all administrator accounts.
2. The system shall require multi-Factor Authentication as a mandatory security feature for all administrator accounts.

3.5.4.4 Data Protection

1. The system shall encrypt all user information for passwords, and payment information.
2. The system shall not store customer payment information directly within the system database.

3.5.5 Maintainability

1. The system shall be designed using a modular monolithic architecture to separate major functional areas into distinct modules.
2. The system shall provide clear and unambiguous documented interfaces for communication between software modules.

3. The system shall allow administrators to update movie listings, showtimes, ticket prices, and theater information without requiring source code modifications.
4. The system shall have clearly documented source code stored in a version control system.
5. The system shall have comprehensive testing at different levels such as unit tests, integration tests and performance tests to support future maintenance.
6. The system shall keep records of all test results.
7. The system shall log system events, performance, and error messages with timestamps and descriptive information to support troubleshooting and maintenance.
8. The system shall include developer and administrator documentation to support software maintenance, and future extensions,

3.5.6 Portability

1. The system shall operate on the latest stable versions of major web browsers:
    a. Chrome
    b. Firefox
    c. Safari
    d. Edge.
2. The system shall provide consistent functionality and user interface behavior across all supported web browsers.
3. The system shall not require users to install additional software or browser plugins to access the system,
4. The system shall ensure that no more than 10% of the application source code is browser-dependent code to support portability across all supported web browsers.
5. The system shall operate independently of the user's operating system when accessed through a supported web browser.

4. Software Design

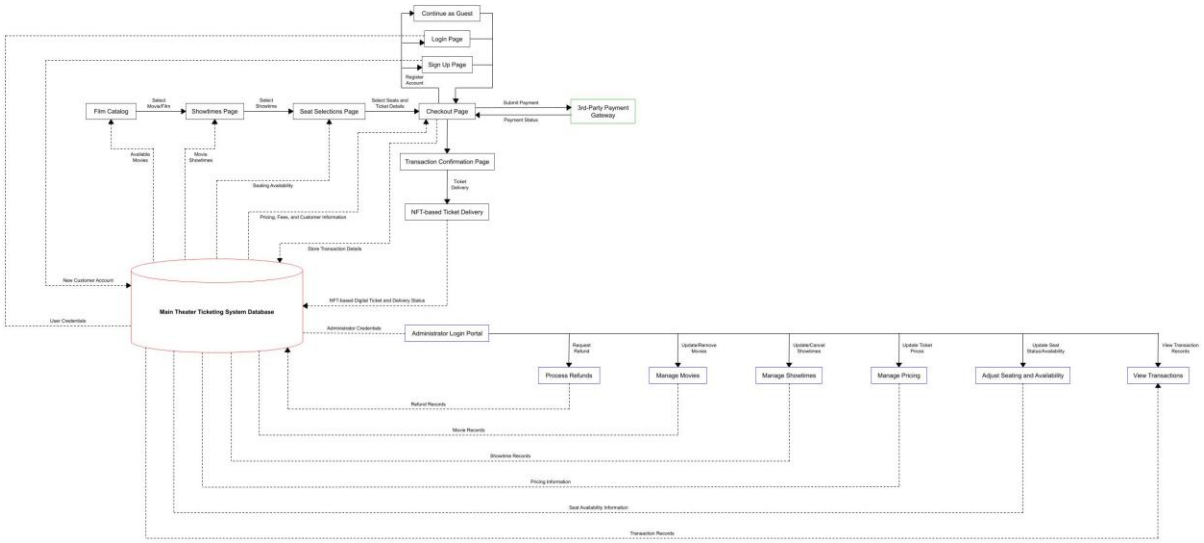
4.1 Software Design Overview

The software design defines the architecture and object-oriented design of the Theater Ticketing System. This is a blueprint for the Theater Ticketing System by translating the functional and non-functional requirements described in section 3 into a technical design for implementation. This includes the major software components, their interactions with each other, and the object-oriented structure of the system.

The software architecture provides the organization and components of the system and communication between its components. The UML Class Diagram provides a detailed description of the system's classes, their attributes, operations, and the relations between classes, creating a foundation for implementation. This section is intended for the software developers, system designers, and maintainers who have roles in implementing, reviewing, and enhancing the Theater Ticketing System.

4.2 Software Architecture Overview

4.2.1 SWA Diagram



4.2.2 SWA Description

The Software Architecture (SWA) of the Theater Ticketing System is designed using a modular architecture that separates the presentation layer, business logic layer, data layer, and external services. This architecture organizes user interactions, application processing, data management, and third-party integrations into distinct components to improve maintainability, scalability, security, and system reliability.

The system begins at the **Film Catalog (Home Page)**, where customers browse the list of available movies retrieved from the **Main Theater Ticketing System Database**. After selecting a movie, customers proceed to the **Showtimes Page** to view available screening schedules. They then navigate to the **Seat Selection Page**, where real-time seat availability is retrieved from the database, allowing customers to select their preferred seats before continuing to the **Checkout Page**.

Before completing the purchase, customers have the option to register a new account, log in to an existing account, or continue as a guest. Registered users are authenticated using credentials stored in the database, while newly registered customer information is saved to the database. Guest users can continue directly to checkout without creating an account.

At the **Checkout Page**, the system reviews the selected seats and ticket details, calculates the total ticket price, applicable taxes, and service fees, and prepares the transaction for payment. The payment request is securely transmitted to the external **Third-Party Payment Gateway**, which authorizes or rejects the transaction. The payment status is then returned to the system to determine whether the purchase is successful.

When payment is successfully authorized, the completed transaction is stored in the database, booking records are finalized, and the customer is redirected to the **Transaction Confirmation**

Page. The system then generates and delivers an **NFT-based digital ticket**, allowing the customer to access the purchased ticket securely.

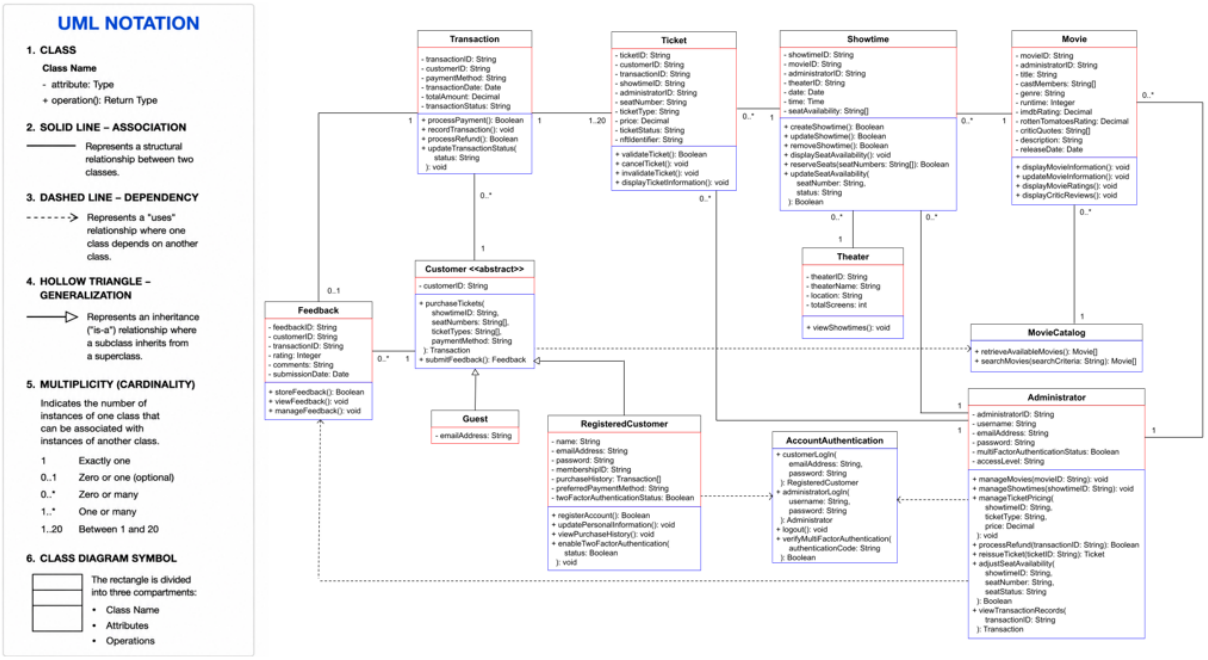
The **Main Theater Ticketing System Database** serves as the central repository for all application data. It stores customer and administrator credentials, movie records, showtime schedules, seat availability information, pricing information, transaction records, refund records, and NFT-based ticket delivery information. All system components retrieve and update information through the database, ensuring data consistency and integrity throughout the application.

The system also includes an **Administrator Login Portal** that allows authorized staff to access administrative functions after successful authentication. Through the portal, administrators can process refunds, manage movies, manage showtimes, update ticket pricing, adjust seat availability, and view transaction records. Any administrative changes are stored in the database and are immediately reflected in the customer-facing components of the system.

Overall, the software architecture provides a modular and secure design by separating customer services, administrative operations, centralized database management, and external payment processing. This separation enhances maintainability, scalability, and system security while supporting future feature enhancements and ensuring a reliable theater ticket purchasing experience.

4.3 UML Class Diagram

4.3.1 UML Diagram



4.3.2 UML Class Description

4.3.2.1 Class: Customer

Class Overview:

The purpose of the Customer class is to represent a general user who interacts with the Theater Ticketing System. The class is an abstract parent class for the Guest and RegisteredCustomer classes. The class contains the common customerID attribute, and common operations such as purchaseTickets() and submitFeedback(). The class attributes and operations are inherited by the Guest and RegisteredCustomer classes.

- **Attributes:**

- customerID (String):

customerID represents the customer identifier, which is unique to each customer. For a Guest, the customerID is temporary. For a RegisteredCustomer, customerID is permanently associated with the customer's account. The customerID is used to identify the customer when the customer purchases tickets or submits feedback in the Theater Ticketing System.

- **Functions:**

- purchaseTickets(showtimeID: String, seatNumbers: String[], ticketTypes: String[], paymentMethod: String): Transaction

The function purchases a movie ticket based on the customer's desired showtime, seat selections, ticket types, and payment method. The function calculates the ticket prices based on the customer type(Guest or RegisteredCustomer), selected ticket categories (regular, student, military, senior) and applicable fees. The function submits the payment request to the Third-Party payment service and verifies the payment. The function uses showTimeID, seatNumbers, ticketTypes, and paymentMethod as parameters to create Ticket objects and a Transaction object which are associated with the customerID after a successful payment. The generated Ticket objects will be recorded in the Transaction object. The function updates the seat availability in the Theater Ticketing System. The function returns the generated Transaction object after a successful ticket purchase. The function displays an error message and exits gracefully if an error occurs.

- submitFeedback(): Feedback

The function asks the customer to provide feedback, which includes a smiley-face rating scale with optional comments. The function also provides an option to skip the submission. The function uses the provided information to create a Feedback object that is associated with the customerID. The function returns the created Feedback object after a successful feedback submission. The function displays an error message and exits gracefully if an error occurs.

4.3.2.2 Class: Guest

Class Overview:

The purpose of the Guest class is to represent a user who does not have a registered account to purchase tickets in the Theater Ticketing System. The class is a subclass of the Customer class and inherits the customerID attribute and the purchaseTickets() and submitFeedback() functions from the Customer class. The Guest class stores the email address for ticket confirmation, delivery, and communication purposes.

- **Attributes**

- emailAddress(String):

emailAddress represents the guest customer's email address which is used to deliver ticket information and NFT-based digital tickets after a successful ticket purchase.

- **Functions**

*The Guest class does not contain additional functions. The class inherits purchaseTickets() and submitFeedback() functions from the Customer class. Guest customers are charged the online service fee and seat reservation fee during the ticket purchase process.

4.3.2.3 Class: RegisteredCustomer

Class Overview:

The purpose of the RegisteredCustomer class is to represent a user who creates a registered account to interact with the Theater Ticketing System. This class is a subclass of the Customer class and inherits the customerID attribute and the purchaseTickets() and submitFeedback() functions from the Customer abstract class. The RegisteredCustomer class stores and manages customer account information such as account login and personal information. It also defines operations related to account registration, updating personal infromation, accessing purchase ticket history, and managing security preferences for Two-Factor Authentication.

- **Attributes**

- name (String):
  name represents the customer's name for the registered account.
- emailAddress (String):
  emailAddress represents the customer's email address for the registered account. It is used for communication and authentication.
- password (String):
  password represents the hashed value of the customer's password. The original plaintext password is not stored directly in the password..
- membershipID (String)
  membershipID represents the customer's membership identification number for future membership or reward program features.
- purchaseHistory (Transaction[])
  purchaseHistory represents the list of Transaction objects that store the previous transactions made by the registered customer account.
- preferredPaymentMethod (String):
  preferredPaymentMethod represents the customer's preferred payment method for purchasing tickets.
- twoFactorAuthenticationStatus (Boolean):
  twoFactorAuthenticationStatus represents the customer's status for enabling Two-Factor Authentication for the registered customer account. The attribute is initially set to False when the RegisteredCustomer object is created. The attribute is set to True if the customer enables Two-Factor Authentication, and False otherwise.

- **Functions**

- registerAccount(): Boolean
  The function creates a new registered account for the customer. The function asks the customer for an email address, password, and name, and stores the information in the system.

The function generates a unique membership identifier and stores it in the membershipID. The function returns True when the account registration process is successful. The function displays an error message and returns False if the registration process is unsuccessful.

- updatePersonalInformation(): void
  The function modifies the customer's personal information such as the customer's name and preferred payment method, according to the customer's desired changes. The function updates the customer's account information in the system after the provided information is validated.

- viewPurchaseHistory(): void
  The function retrieves the customer's purchaseHistory, and displays the list of previous transactions and the associated NFT-based digital tickets for the registered customer.

- enableTwoFactorAuthentication(stat: Boolean): void
  The function enables or disables the Two-Factor Authentication based on the customer's preference. The function accepts stat as the input Boolean parameter. If stat is True, the function enables Two-Factor Authentication, and if stat is False, it disables Two-Factor Authentication. The function updates the twoFactorAuthenticationStatus in the system.

* The RegisteredCustomer class inherits purchaseTickets() and submitFeedback() functions from the Customer class. The online service fee and seat reservation fee are waived for registered customers during the ticket purchase process.

4.3.2.4 Class: Administrator

Class Overview:

The purpose of the Administrator class is to represent an authorized theater employee who manages the Theater Ticketing System through the secure administrative interface. The Administrator class stores administrator account information such as account login information, contact information, Multi-Factor Authentication status, and access level. The class defines operations related to managing movies, showtimes, and ticket pricing, processing refunds, reissuing tickets, adjusting seat availability, and viewing transaction records.

- **Attributes**

- administratorID (String):
  administratorID represents the administrator identifier, which is unique to each administrator. It is used to identify the administrator during administrator operations.
- username (String):
  username represents the administrator's username for the administrator account. It is used to log into the administrator account during the authentication process.
- emailAddress (String):
  emailAddress represents the administrator's email address that is associated with the administrator account. It is provided by the theater management and is not the administrator's personal email address. It is used for communication and Multi-Factor authentication.
- password (String):
  password represents the hashed value of the administrator's password for account login. The original password shall not be stored directly in the password.
- multiFactorAuthenticationStatus (Boolean):

multiFactorAuthenticationStatus represents the administrator's status for setting up Multi-Factor Authentication for the administrator account. The attribute is initially set to False when the administrator object is created. The attribute is set to True if the administrator successfully set up Multi-Factor Authentication, and False otherwise.

- accessLevel (String):

accessLevel represents the administrator's level of access for the administrator account. This attribute is initially set to "Level 1" and is intended for future enhancements. It is designed to restrict administrative operations based on the administrator's access level.

• **Functions**

- manageMovies (movieID: String): void

manageMovies manages movie information in the Theater Ticketing System. The function uses the movieID as a parameter to identify the movie the administrator wants to manage. The administrator can add a new movie, update the existing information, or remove a movie from the system. The function displays an error message and exits gracefully if an error occurs.

-manageShowtimes(showtimeID: String): void

The function manages the showtime information in the Theater Ticketing System. The function uses the showtimeID as a parameter to identify the showtime that the administrator wants to manage. The administrator can create a new showtime, update existing showtime information, or remove a showtime from the system. The function displays an error message and exits gracefully if an error occurs.

- manageTicketPricing(showtimeID: String, ticketType: String, price: Decimal): void

The function manages the price of the ticket in the Theater Ticketing System. The function uses the showtimeID and ticketType from the parameter list to identify the showtime and ticket category that the administrator wants to change. The function sets the price of the ticket to the value provided by the price in the parameter list. The administrator can update the ticket price for the selected showtime and ticket category. The function displays an error message and exits gracefully if an error occurs.

- processRefund(transactionID: String): Boolean

The function processes a refund for a completed transaction. The function uses the transactionID parameter to identify the transaction that the administrator wants to refund. The function also verifies that the transaction is eligible to process a refund. If it is eligible, the refund request is submitted to the Third-Party Payment Service. The function updates the transaction status and returns True if the refund request is approved. If the refund process is unsuccessful, the function displays a message and returns False.

- reissueTicket(ticketID: String): Ticket

The function reissues a ticket. The function uses the ticketID parameter to identify the ticket that the administrator wants to reissue. The function retrieves the Ticket object based on the ticketID and reissues the ticket to the customer. The function returns the retrieved Ticket object. This function is intended for customers who lose their ticket. The function displays an error message and exits gracefully if the Ticket object cannot be found.

- adjustSeatAvailability(showtimeID: String, seatNumber: String, seatStatus: String): Boolean

The function adjusts seat availability in the Theater Ticketing System. The function uses the showtimeID and seatNumber parameters to identify the seat that the administrator wants to adjust. The function can block or unblock the selected seat based on the seatStatus parameter. The function returns True if the seat availability is successfully updated. The function cannot change the seat availability if the seat is already occupied. The function displays a message and returns False if the seat availability cannot be updated.

- viewTransactionRecords(transactionID: String): Transaction

The function views a transaction record in the Theater Ticketing System. The function uses the transactionID parameter to identify the transaction record that the administrator wants to view. The function retrieves the Transaction object based on the transactionID, displays the transaction record stored in the retrieved Transaction object, and returns the retrieved Transaction object. The function displays an error message and exits gracefully if the Transaction object cannot be found.

4.3.2.5 Class: AccountAuthentication

Class Overview:

The purpose of the AccountAuthentication class is to authenticate customer and administrator credentials in the Theater Ticketing System. The class defines the operations related to customer login, administrator login, logout, and Multi-Factor Authentication verification.

- **Attributes**

- None

- **Functions**

- customerLogIn(emailAddress: String, password: String): RegisteredCustomer

The function authenticates a registered customer account using the emailAddress and password inputs. The function finds the matching emailAddress stored in the Theater Ticketing system database and retrieves the associated registered customer account. The function verifies the password parameter against the stored hashed password in the retrieved registered customer account. If the emailAddress is found and the password verification is successful, the function returns the corresponding RegisteredCustomer object. The function displays a generic message without describing which credential is incorrect and exits gracefully if authentication is unsuccessful.

- administratorLogIn(username: String, password: String): Administrator

The function authenticates an administrator account using the username and password inputs. The function finds the matching username stored in the Theater Ticketing system database and retrieves the associated administrator account. The function verifies the password parameter against the stored hashed password in the retrieved administrator account. If the username is found and the password verification is successful, the function returns the corresponding Administrator object. The function displays a generic message without describing which credential is incorrect and exits gracefully if authentication is unsuccessful.

- logout(): void

The function terminates the current user session for either a registered customer or administrator. The function logs the user out of the active session and returns the user to the appropriate login page completing the logout process. The function displays an error message and exits gracefully if the logout process is unsuccessful.

- verifyMultiFactorAuthentication(authenticationCode: String): Boolean
  The function verifies the Multi-Factor Authentication code provided by the user during the login process. The function compares the provided authenticationCode parameter with the authentication code generated by the Theater Ticketing System. This function is intended for both registered customers and administrators. The function returns True if the verification process is successful and False if the verification process fails.

4.3.2.6 Class: Movie

Class Overview:

The purpose of the Movie class is to represent a movie in the theater. The Movie class stores movie information, ratings and critic reviews for the movie. The Movie class also defines operations related to displaying movie information, updating movie information, and displaying movie ratings and critic reviews.

- **Attributes**

- movieID (String):
  movieID represents the movie identifier, which is unique to each movie. It is used to identify the movie in the Theater Ticketing System.
- administratorID (String):
  administratorID represents the unique identifier of an administrator. It is used to identify the administrator who last modified the Movie object in the Theater Ticketing System.
- title (String):
  title represents the title of the movie.
- castMembers (String[]):
  castMembers represents the list of cast members, such as actors and actresses, in the movie.
- genre (String):
  genre represents the genre of the movie, such as action, comedy, drama, horror, and others.
- runtime (Integer):
  runtime represents the total duration of the movie in minutes.
- imdbRating (Decimal):
  imdbRating represents the rating of the movie obtained from IMDb.
- rottenTomatoesRating (Decimal):
  rottenTomatoesRating represents the rating of the movie obtained from Rotten Tomatoes.
- criticQuotes (String[]):
  criticQuotes represents the list of selected quotes from critics who reviewed the movie.
- description (String):
  description represents a brief summary of the movie.
- releaseDate (Date):
  releaseDate represents the official release date of the movie.

- **Functions**

- displayMovieInformation(): void
  The function displays the movie information stored in the Movie object. The function displays the movieID, title, cast members, genre, runtime, description, and release date. The function displays “Not available at the moment” for the corresponding information if any of the information is unavailable.

- updateMovieInformation(): void
  The function updates the movie information stored in the Movie object. The administrator is asked which movie information to update, and the function updates the information based on the administrator’s input. The function can update the movie title, cast members, genre, runtime, description, release date, IMDb rating, Rotten Tomatoes rating, and critic quotes. The updated information is stored in the Theater Ticketing System database. The function displays an error message and exits gracefully if an error occurs.

- displayMovieRatings(): void
  The function displays the ratings for the movie. The function displays both the IMDb rating and the Rotten Tomatoes rating stored in the Movie object. The function displays “Not available at the moment” for the corresponding rating if a rating is unavailable.

- displayCriticReviews(): void
  The function displays the critic quotes associated with the movie. The function displays the list of selected critic quotes stored in the Movie object. The function displays “Not available at the moment” if the critic quotes are unavailable.

4.3.2.7 Class: Showtime

Class Overview:

The purpose of the Showtime class is to represent a scheduled movie showing in the Theater. The Showtime class stores information related to the scheduled showing, such as the showtime ID, moive ID, theater location, date, time, and seat availability. The class also defines operations related to managing scheduled showing, displaying seat availability, reserving seats, and updating seat availability.

- **Attributes**

- showtimeID (String):
  showtimeID represents the showtime identifier which is unique to each scheduled movie showing. It is used to identify the scheduled movie showing in the Theater Ticketing System.
- movieID (String):
  movieID represents the movie identifier that is associated with the scheduled movie showing. It is used to identify the movie being shown for the scheduled movie showing.
- theaterID (String):
  theaterID represents the theater identifier that is associated with the scheduled movie showing. It is used to identify the theater being shown for the scheduled movie showing.
- administratorID (String):
  administratorID represents the unique identifier of an administrator. It is used to identify the administrator who last modified the Showtime object in the Theater Ticketing System.
- showDate (Date):
  showDate represents the scheduled date of the movie showing.

- time (Time):
  time represents the scheduled time of the movie showing.
- seatAvailability (String[]):
  seatAvailability represents the list of seat statuses for the scheduled movie showing. The status of each seat includes available, sold, reserved, blocked, or unavailable. When a new Showtime object is created, the initial status of all seats is set to available. Each array index represents a seat number, and the value stored at each index represents the status of that seat.

• **Functions**
- createShowTime(): Boolean
  The function creates a new scheduled movie showing in the Theater Ticketing System. The administrator is asked to enter the movie ID, theater location, show date and time, and seat availability. The function creates a new Showtime object using the provided information and stores it in the Theater Ticketing System database. The function returns True if the new scheduled movie showing is successfully created and False if the creation process fails.

- updateShowtime(): Boolean
  The function updates the information of an existing scheduled movie showing in the Theater Ticketing System. The administrator is asked to select the information to update. The function can update the movie ID, theater location, show date and time, and seat availability of the Showtime object. The updated information of the Showtime object is stored in the Theater Ticketing System database. The function returns True if the update process is successfully completed and False if the process fails.

- removeShowtime(): Boolean
  The function removes an existing scheduled movie showing from the Theater Ticketing System. The administrator is asked to enter the showtimeID of the scheduled movie showing to remove. The function searches for the corresponding Showtime object and deletes the object from the Theater Ticketing System database. The function returns True if the removal process is successful and False if the corresponding Showtime object cannot be found or the removal process fails.

- displaySeatAvailability(): void
  The function displays the seat availability of the scheduled movie showing. The function retrieves the seatAvailability information stored in the Showtime object and displays the status of all seats.

- reserveSeats(seatNumbers: String[]): Boolean
  The function reserves the selected seats for the scheduled movie showing. The function checks whether the selected seats are available. If all seats are available, the function updates the status of the seats to “reserved” using the seatNumbers parameter and stores the updated seat availability information in the Theater Ticketing System database. The function returns True if all selected seats are successfully reserved and False if the reservation process fails.

- updateSeatAvailability(seatNumber: String, status: String): Boolean
  The function updates the seat status for the scheduled move showing. The function searches for the seat using the seatNumber parameter and updates the status of the seat based on the status

parameter. The updated seat status is stored in the Theater Ticketing System database. The function returns True if the seat availability is successfully updated and False if the update process fails.

4.3.2.8 Class: MovieCatalog

Class Overview:

The purpose of the MovieCatalog class is to provide the functionality for retrieving and searching movies in the Theater Ticketing System. The class defines operations related to retrieving and displaying the list of available movies and searching for movies based on the provided search criteria. The MovieCatalog class does not store Movie objects or their information. The class retrieves Movie objects from the Theater Ticketing System database.

- **Attributes**

- None

- **Functions**

- retrieveAvailableMovies(): Movie[]

The function retrieves the list of all available movies from the Theater Ticketing System database. The function stores the retrieved Movie objects in a list and displays all Movie objects. The function returns the list of retrieved Movie objects. The function displays an error message and exits gracefully if any database error occurs.

- searchMovies(searchCriteria: String): Movie[]

The function searches for available movies in the Theater Ticketing System database. The function searches for movies using the searchCriteria parameter. The searchCriteria parameter can include the title of the movie, genre, and cast members. The function retrieves the Movie objects that match the searchCriteria parameter and stores them in a list. The function returns the list of retrieved Movie objects. The function displays an error message and exits gracefully if any database error occurs.

4.3.2.9 Class: Ticket

Class Overview:

The purpose of the Ticket class is to represent a movie ticket purchased in the Theater Ticketing System. The Ticket class stores the ticket ID, customer ID, showtime ID, seat number, ticket type, price, ticket status, and NFT identifier. The class defines operations related to validating tickets, ticket cancellation, invalidating tickets, and displaying ticket information.

- **Attributes**

- ticketID (String):

ticketID represents the unique identifier of a ticket purchased in the Theater Ticketing System. It is used to identify the ticket and manage the ticket for validation, cancellation, and invalidation.

- customerID (String):

customerID represents the unique identifier of the customer who purchased the ticket in the Theater Ticketing System. It is associated with the ticket after a successful ticket purchase.

- transactionID (String):

transactionID represents the unique identifier of a transaction in the Theater Ticketing System. It is used to associate the transaction with one or more purchased tickets after a successful ticket purchase.
- showtimeID (String):
showtimeID represents the unique identifier of the showtime associated with the ticket in the Theater Ticketing System. It is used to identify the scheduled movie showing associated with the ticket.
- administratorID (String):
administratorID represents the unique identifier of an administrator. It is used to identify the administrator who last modified the Ticket object in the Theater Ticketing System.
- seatNumber (String):
seatNumber represents the seat number on the ticket. It identifies the seat assigned to the customer for the scheduled movie showing.
- ticketType (String):
ticketType represents the type of ticket purchased in the Theater Ticketing system. The ticketType can be regular, student, military, or senior.
- price (Decimal):
price represents the price of the ticket. The price of the ticket is based on the ticketType.
- ticketStatus (String):
ticketStatus represents the status of the ticket in the Theater Ticketing System. The ticketStatus can be valid, invalid or refunded.
- nftIdentifier (String):
nftIdentifier represents the unique identifier of the NFT associated with the ticket in the Theater Ticketing System. It is used to identify the NFT-based digital ticket.

• **Functions**
- validateTicket(): Boolean
The function validates the ticket in the Theater Ticketing System. The function checks whether the ticket is valid for admission by checking the status of the ticket. The function returns True if the ticket is successfully validated and False if the ticket is not valid.

- cancelTicket(): void
The function cancels the existing ticket in the Theater Ticketing System. The function updates the ticketStatus to refunded and processes the ticket cancellation. The function updates the seat availability by making the corresponding seat available so that it can be purchased by another customer. The function is intended for processing refunds by an authorized administrator. The function also stores the canceled ticket information in the Theater Ticketing System database for further review.

- invalidateTicket(): void
The function invalidates the ticket in the Theater Ticketing System. The function updates the ticketStatus to invalid after the ticket is used for admission. The updated ticket status is stored in the Theater Ticketing System database.

- displayTicketInformation(): void
The function displays the ticket information in the Theater Ticketing System. The function retrieves the associated ticket information, such as ticketID, customerID, showtimeID,

seatNumber, ticketType, ticketStatus, nftIdentifier, and price. The function displays all the retrieved information.

4.3.2.10 Class: Transaction

Class Overview:

The purpose of the Transaction class is to represent a ticket purchase transaction in the Theater Ticketing System. The class stores transaction information, such as the transaction ID, customer ID, tickets, payment method, transaction date, total amount, and transaction status. The class defines operations related to processing payments, recording transaction information, processing refunds, and updating transaction status.

- **Attributes**

- transactionID (String):
  transactionID represents the identifier of the transaction which is unique to each transaction in the Theater Ticketing System. It is used to identify the ticket purchase transaction.
- customerID (String):
  customerID represents the unique identifier of the customer associated with the transaction in the Theater Ticketing System. It is used to identify the customer who completed the ticket purchase transaction.
- paymentMethod (String):
  paymentMethod represents the payment method used to complete the ticket purchase transaction in the Theater Ticketing System. The payment method can be Visa, MasterCard, American Express (with the associated last 4 digits of the card number), or PayPal.
- transactionDate (Date):
  transactionDate represents the date when the ticket purchase transaction was completed in the Theater Ticketing System.
- totalAmount (Decimal):
  totalAmount represents the total amount charged to the customer in the transaction in the Theater Ticketing System. It is calculated based on all tickets purchased in a single transaction.
- transactionStatus (String):
  transactionStatus represents the status of the transaction in the Theater Ticketing System. The transactionStatus can be completed, failed, or refunded.

- **Functions**

- processPayment(): Boolean
  The function processes the payment for the ticket purchase transaction in the Theater Ticketing System. The function verifies the payment information, submits the payment request to the Third-Party Payment Service, and verifies the response returned by the Third-Party Payment Service. The function updates the corresponding attributes, such as the customerID, tickets, paymentMethod, transactionDate, totalAmount, and transactionStatus, in the Theater Ticketing System database. The function sets transactionStatus to completed or failed based on the response returned by the Third-Party Payment Service. The function returns True if the payment is successfully processed and False if the payment fails.

- recordTransaction(): void

The function records the ticket purchase transaction in the Theater Ticketing System. The function stores the current transaction information of the Transaction object, such as transactionID, customerID, tickets, paymentMethod, transactionDate, totalAmount, and transactionStatus, in the Theater Ticketing System database. The function is intended for future review.

- processRefund(): Boolean

The function processes the refund for the ticket purchase transaction in the Theater Ticketing System. The function submits the refund request using the payment method the customer used when the ticket was purchased to the Third-Party Payment Service and verifies the response returned by the Third-Party Payment Service. The function sets transactionStatus to refunded and updates transactionStatus in the Theater Ticketing System database. The function returns True if the refund is successfully processed and False if the refund process fails.

- updateTransactionStatus(status: String): void

The function updates the transaction status in the Theater Ticketing System. The function updates the transactionStatus based on the status parameter. The function stores the updated transactionStatus in the Theater Ticketing System database.

4.3.2.11 Class: Feedback

Class Overview:

The purpose of the Feedback class is to represent feedback submitted by customers in the Theater Ticketing System. Customers can submit feedback after a successful ticket purchase. The class stores feedback information, such as the feedback ID, customer ID, transaction ID, rating, comments, and submission date. The class defines operations related to storing feedback in the Theater Ticketing System database, viewing feedback for review by administrators, and deleting feedback after review.

- **Attributes**

- feedbackID (String):

feedbackID represents the identifier of the feedback which is unique to each feedback in the Theater Ticketing System. It is used to identify and manage customer feedback.

- customerID (String):

customerID represents the unique identifier of the customer who submitted the feedback in the Theater Ticketing System. It is used to identify the customer associated with the feedback.

- transactionID (String):

transactionID represents the unique identifier of the ticket purchase transaction associated with the feedback in the Theater Ticketing System.

- rating (Integer):

rating represents the customer's rating for the experience of using the Theater Ticketing System. The rating ranges from 1 to 5 based on the smiley-face satisfaction rating scale. The rating is used to evaluate the customer's overall experience.

- comments (String):

comments represents the customer's written feedback about their experience using the Theater Ticketing System.

- submissionDate (Date):

submissionDate represents the date when the customer submitted the feedback in the Theater Ticketing System.

• **Functions**
- storeFeedback(): Boolean
The function stores the customer feedback in the Theater Ticketing System. The function verifies the feedback information, such as customerID, transactionID, rating, comments, and submissionDate, and stores the feedback information in the Theater Ticketing System database. The function returns True if the feedback is successfully stored and False if an error occurs or the feedback cannot be stored.

- viewFeedback(): void
The function displays customer feedback in the Theater Ticketing System. The function retrieves the feedback information, such as feedbackID, customerID, transactionID, rating, comments, and submissionDate, from the Theater Ticketing System database. The function displays the retrieved feedback information for review by administrators.

- manageFeedback(): void
The function manages customer feedback in the Theater Ticketing System. The function allows administrators to review customer feedback by customerID and delete customer feedback after review. The function records all changes and updated feedback information in the Theater Ticketing System database.

4.3.2.12 Class: Theater
Class Overview:
The purpose of the class is to represent a physical theater location where movies are shown. The class stores theater information, such as theaterID, location, and totalScreens. The class defines an operation for viewing the showtimes scheduled at the theater.

• **Attributes**
- theaterID (String):
theaterID represents the identifier of the theater which is unique to each theater in the Theater Ticketing System. It is used to identify and manage theater.
- theaterName (String):
theaterName represents the name of the theater.
- location(String):
location represents the location of the theater where the scheduled movie showing takes place.
- totalScreens(Integer):
totalScreens represents the total number of screens available at the theater.

• **Functions**
- viewShowtimes(): void
The function displays all showtimes scheduled at the theater in the Theater Ticketing System. The function retrieves and displays showtime information, such as showtimeID and movieID.

5. Analysis Models

Overview:

This section provides the analysis models used to describe the behavior of the Theater Ticketing System. These models complement the software architecture and UML class design presented in Section 4 by showing how system components interact with each other to satisfy the functional requirements presented in Section 3. Each analysis model includes an introduction, a narrative description, and a diagram.

5.1 Purchase Ticket Sequence Diagram

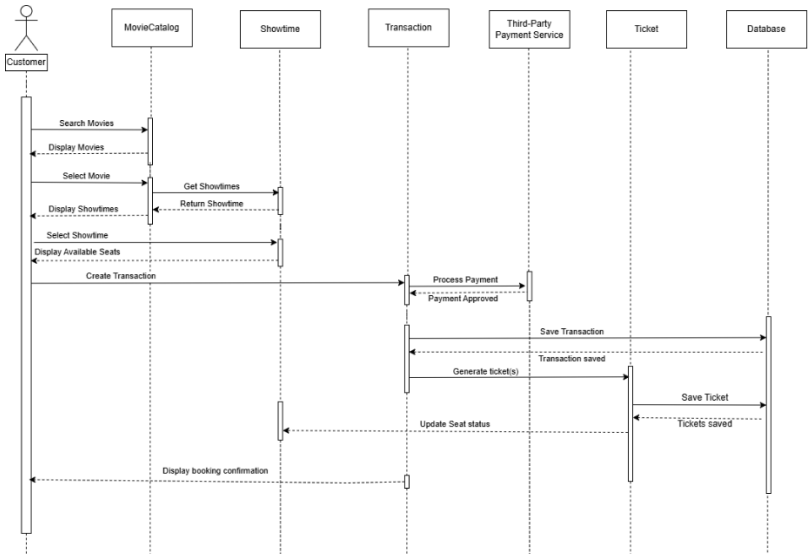
Introduction:

The Purchase Ticket Sequence Diagram illustrates the behavior of the Theater Ticketing System when the customer purchases a ticket. The diagram illustrates the interactions between the customer, system components, third-party payment service, and the database during the ticket purchase process. The diagram begins with movie selection and ends with digital ticket delivery after successful payment. This sequence diagram supports the Ticket Purchase functional requirement presented in Section 3.2.1 of this SRS.

Narrative Description:

The sequence begins when the customer searches for and selects a movie from the Movie Catalog. The system retrieves the selected movie information, availability showtimes, and seat availability. After the customer selects the preferred showtime and seat, the system creates the transaction, displays the checkout page, and calculates the ticket price. The customer then proceeds with payment, and the system submits the payment request to the third-party payment service for verification. After the payment is approved, the system records the transaction in the database, generates the NFT-based digital ticket, records the ticket information in the database, and updates the seat availability. The system then displays the purchase confirmation page, and finally delivers the digital ticket to the customer.

Sequence Diagram:



Sequence Diagram for Ticket Purchase by Customer

5.2 Administrator Refund Sequence Diagram

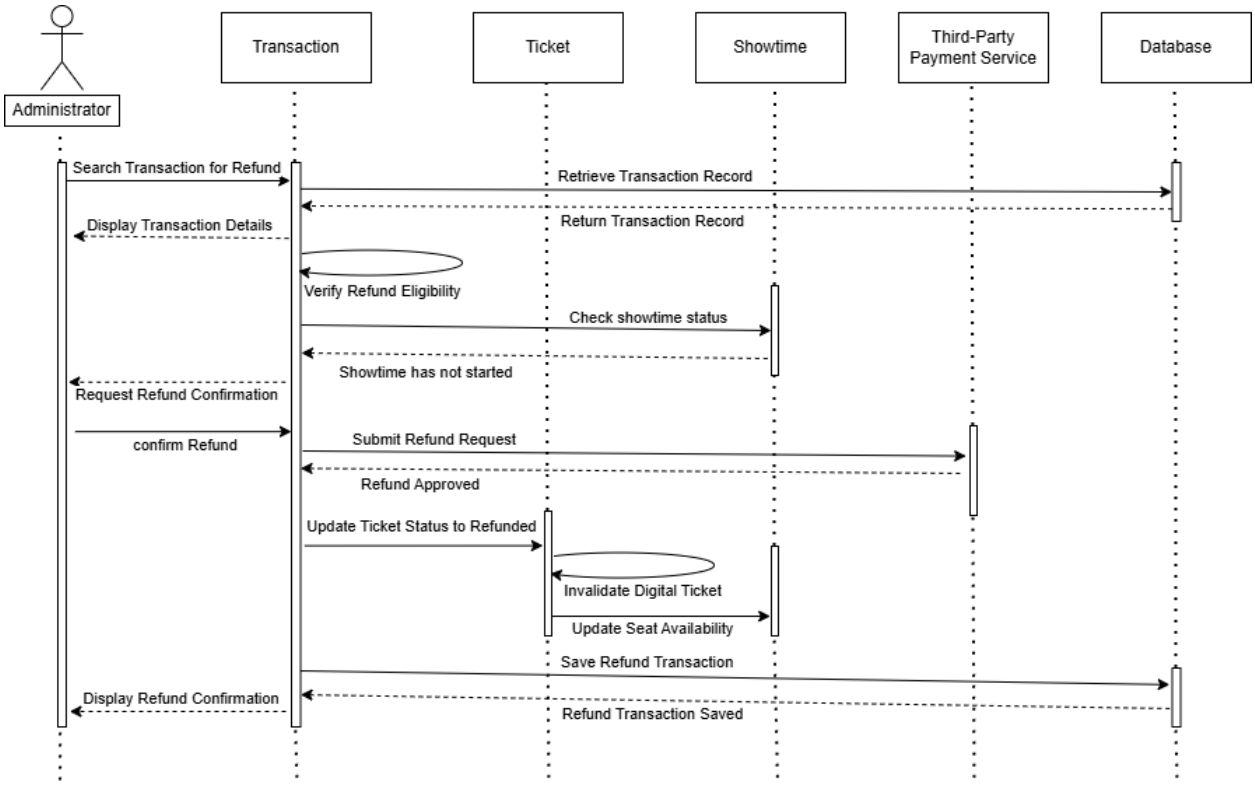
Introduction:

The Administrator Refund Sequence Diagram illustrates the behavior of the Theater Ticketing System when an administrator processes a customer refund. The diagram illustrates the interactions between the administrator, system components, third-party payment service, and the database during the refund process. The diagram begins when the administrator searches for a transaction and ends when the refund confirmation is displayed. This sequence diagram supports the Administrator Management functional requirement presented in Section 3.2.2 of this SRS.

Narrative Description:

The sequence begins when the administrator searches for a transaction to refund. The system retrieves the transaction record from the database and displays the transaction details to the administrator. The system then verifies the refund eligibility by checking that the showtime has not started. If the transaction is eligible for a refund, the system requests confirmation from the administrator before submitting the refund request to the Third-Party Payment Service. After the refund is approved, the system updates the ticket status to refunded and invalidates the digital ticket. The system then updates the seat availability and saves the refund transaction record in the database. At the end, the system displays the refund confirmation to the administrator.

Sequence Diagram:



Sequence Diagram for Refund Process by an Administrator

6. PROJECT MANAGEMENT

6.1 Software System Overview

The Theater Ticketing System is a web-based application developed to provide customers with a convenient and secure way to purchase movie tickets. The system allows customers to browse movies, search available showtimes, select seats, make secure online payments, and receive digital or printed tickets. It also provides administrators with tools to manage movies, showtimes, ticket pricing, seat availability, customer transactions, and refunds. The system uses a centralized database and integrates with a third-party payment gateway to ensure accurate ticket management and secure transaction processing. The following sections summarize the project's development tasks, estimated timeline, and team member responsibilities.

6.2 Project Tasks

Task	Description
Requirements Analysis	Gather and document the functional and non-functional requirements.
SRS Documentation	Prepare and organize the Software Requirements Specification (SRS).
Use Case Development	Create use case diagrams and detailed use case descriptions.
Software Architecture Design	Design the Software Architecture (SWA) diagram and describe the system components and connectors.
UML Class Design	Develop the UML class diagram and class descriptions.
Analysis Model Development	Create sequence diagrams and other analysis models.
Design Review and Validation	Review and verify the requirements, diagrams, and design documents for completeness and consistency.
Documentation Revision	Revise, proofread, and improve the formatting and organization of the SRS document.
Final Report Preparation	Finalize the project report and prepare it for submission.

6.3 Estimated Project Timeline

Week	Activities
Week 1	Gather project requirements, analyze stakeholder needs, and begin the Software Requirements Specification (SRS).
Week 2	Complete the SRS, create use cases, design the software architecture (SWA), and develop the UML class diagram.

Week 3	Review and verify the system design, perform testing and validation of the requirements, UML, and SWA diagrams, and revise the documentation based on feedback.
Week 4	Refine the software architecture, database design, and system components. Update diagrams and documentation as needed.
Week 5	Review security, networking, and system architecture considerations. Finalize all design artifacts and documentation.
Week 6	Complete the final report, conduct the final project review, proofread the document, and prepare the project for submission.

6.4 Team Responsibilities

Team Member	**Responsibilities**
Trinh Ho	Documentation Revision, Design Review and Validation, Final Report Preparation, and SRS Documentation.
William Y. Tong	Requirements Analysis, Use Case Development, Analysis Model Development, and contributed to Design Review and Validation.
Amina Bendjennat	Software Architecture Design, UML Class Design, SRS Documentation, and contributed to Final Report Preparation with Trinh Ho.

7. Links to GitHub

Trinh Ho:
https://github.com/tho7293-droid/CS250-Theater-Ticketing-System-
William Y Tong:
https://github.com/WilliamYTong/CS250_TheaterTicketingSystem_GP10_WilliamYTong
Amina Bendjennat:
https://github.com/abendjennat3240-cmyk/ABendjennat-CS250-SRS-Part2.git

8. Testing

The main purpose of this test plan is to verify and confirm that all requirements stated in this Software Requirements Specification are satisfied by the Theater Ticketing System. Testing is performed at a variety of levels to validate components of the software individually as well as their interactions with one another and overall system functionality. Test cases are created in order to check the success of ticket purchasing, customer and administrator account

management, payment systems, seating reservations, and customer feedback

The UML and SWA diagrams were referenced in the creation of this test plan. After reviewing the planned software testing, no modifications were necessary to the design. The diagrams from Assignment 2 fully supports the testing activities that occur in this section.

8.1 Test Cases

The details of the test cases utilized for this Theater Ticketing System are provided in the **Excel Test Case** document submitted alongside this SRS. The document consists of full test approaches labeled by test case IDs with sections for test level, components, priority, test summary, prerequisites, test steps, and expected and actual results.

Unit, integration, functional, and system testing are utilized for the Theater Ticketing System to certify that all requirements are satisfied and the software functions correctly.

8.1.1 Unit Testing

Unit testing checks whether software components and methods function as intended on their own prior to integration with other parts of the system. Tests are included for customer and administrator authentication, the generation of tickets, and validity of customer transactions. These specific trials are utilized to confirm that every component returns the desired outcome when tested with both valid and invalid input information.

8.1.2 Integration Testing

Integration testing checks whether software components interact with one another accurately. Integration tests are included for combinations of payment processing and transaction history, refund processing and ticket status, payment processing and ticket generation, and ticket generation with updates to seat availability. These specific tests double-check that connected parts correctly exchange information with one another while the remainder of the system components retain their original synchronization and values.

8.1.3 Functional Testing

Functional testing checks whether the system satisfies functional requirements specified from the customer's perspective of the application. These tests examine customer account registration with both valid and invalid input credentials, movie searches, and ticket purchasing for both customer and guest users. These specific trials are utilized to confirm that the system returns the desired outcome when tested with both valid and invalid input information from the user.

8.1.4 System Testing

System testing checks whether the system fully operates as intended from start to finish by utilizing a test environment resembling the production system. The test cases examine the complete actions of ticket purchasing, administrator tasks like movie management and refund

processing, and the movie search to ticket purchase procedures. These tests confirm that all components of the system work together as intended and satisfy the complete requirements.

8.1.5 Test Case Summary

The Excel Test Case document consists of 16 example test cases including:

- 4 Unit test cases
- 4 Integration test cases
- 4 Functional test cases
- 4 System test cases

These cases check the main requirements of the Theater Ticketing System which includes customer account management, administrator operations, ticket generation, payment operations and transactions, administrator operations, and overall system functionality.

8.2 Test Environment

The Theater Ticketing System will be tested throughout the development process and in a dedicated testing environment using the Theater Ticketing System web application. During development, **unit, integration, functional, and system testing** will be performed using both manual and automated test cases. The system will also be tested using supported web browsers to verify the functionality of the entire system. A **test database** containing sample customer and administrator accounts, movies, showtimes, seats, and transaction records will be used during testing. External services, including a **third-party payment service**, will also be tested to verify interactions between system components without processing real payments.

The testing environment includes:

- Supported web browsers (Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari)
- Simulated theater kiosk interface
- Test database containing sample customer and administrator accounts
- Sample movies, showtimes, seats, and transaction records
- Test payment gateway

8.3 Test Data

The test cases use both valid and invalid data to verify system functionality, input validation, system behavior, and error handling. Test data includes customer credentials, administrator credentials, movie information, **showtime information**, seat selections, payment information, and transaction information. The Theater Ticketing System will be connected to a test database and executed in both the development and testing environments using **manual testing**. Automated testing will also be used where applicable.

The test data includes:

- Valid and invalid customer login credentials

- Valid and invalid administrator login credentials
- Ticket quantities of **1–20 (valid)** and **21, 22, and 23 (invalid)**
- Valid and invalid transaction IDs
- Valid and invalid payment information
- Valid and invalid movie and showtime information
- Valid and invalid refund requests
- Valid and invalid customer feedback submissions

8.4 Entry and Exit Criteria

Entry Criteria

Testing will begin after the following conditions have been met:

- The required software units have been implemented and are ready for testing.
- The testing environment has been prepared and is operational.
- The test database has been created and populated with sample data.
- The required test cases have been prepared.
- All prerequisite conditions listed in the Excel Test Case document have been satisfied.

Exit Criteria

Testing will be considered complete after the following conditions have been met:

- All test steps listed in the Excel Test Case document have been completed.
- All planned test cases have been executed.
- Any issues encountered during testing have been documented in the Excel Test Case document.
- The actual results have been recorded in the **Actual Result** section of the Excel Test Case document and compared with the expected results.
- The test results demonstrate that the system satisfies the specified requirements, and the outcome of each executed test case has been recorded in the **Status** section of the Excel Test Case document.
- The name of the tester has been recorded in the **Test Executed By** section of the Excel Test Case document.

9. Data Management

9.1 Database Architecture

The Theater Ticketing System uses a relational SQL database to manage, store, manipulate, and control the persistent data required by the system. A centralized database provides consistent data management, maintains data integrity, supports reliable communication among the system components, and reduces data redundancy. This also prevents unnecessary data duplication and

makes sure that all users access the most up-to-date information in the Theater Ticketing System. The database also serves as the central storage for customer accounts, administrator accounts, theater information, movie and showtime information, seat availability, tickets, transactions, and customer feedback.

In this architecture, the web-based application interacts with the centralized database through the software components defined in the Software Architecture (SWA) diagram. Components such as the Authentication service, Theater Management, Movie Management, Ticket Management, Transaction Management, and Feedback Management, are responsible for creating, deleting, retrieving, and updating data according to user requests. In the database, customer and administrator account information is stored to support account authentication and management. Movie and theater information is stored to provide users with currently available movies, showtimes, and theater information. Ticket and transaction records are stored to support ticket purchases, refunds, purchase history, and NFT-based tickets. Customer feedback is also stored to support administrators' review of user comments and improve overall service. Each component interacts with the database only when it is necessary to complete its responsibilities. This architecture makes the system easier to maintain and extend in the future.

To maintain data consistency, the database supports relationships between the system entities. These relationships reduce redundant data storage because related data is stored once and referenced across the system. As a result, modifications to shared data are reflected throughout the system making it easier to maintain and update information. The database also ensures that data remains persistent and available at any time.

External services, which are the payment gateway and email service, will not directly store data in or access the database. Instead, they communicate with the web-based application. The payment gateway processes the payment request and returns the transaction result to the application, and the email service sends confirmation messages with generated digital tickets after a successful purchase. As a result, this prevents external services from accessing the database and improves the system's security. All database operations are managed by the web-based application.

The following Section 9.2 provides a detailed view of how the application components interact with the centralized database and presents the entity relationships that define the database structure of the Theater Ticketing System.

9.2 Data Organization

The Theater Ticketing System organizes data into related entities in the SQL database. Each entity represents a specific component of the system, and these entities are connected to their related entities. These connections maintain the data consistency, and support the application in retrieving and updating data efficiently. The following section shows the database tables and the Entity Relationship diagram that presents the organization of the database.

9.2.1 Database Tables

9.2.1.1 Table: Movie

ID	title	Cast Members	genre	runtime	Imdb Rating	Rotten tomatoes Rating	Critic Quotes	description	Release date	**administrator ID**
00001	Interstellar	Matthew McConaughey, Anne Hathaway	Science Fiction	169	8.7	73	"Gorgeous, heartbreaking epic"	"Astronauts travels through a wormhole"	2014-11-07	1
00002	Titanic	Leonardo DiCaprio, Kate Winslet	Drama	194	7.9	88	"A timeless cinematic achievement."	"Two young lovers from different backgrounds"	1997-12-19	1
00003	Spider-Man	Tobey Maguire, Kirsten Dunst	Action	121	7.4	90	"The superhero film that started a new era."	"Spider-like abilities and becomes the superhero"	2002-05-03	2

9.2.1.2 Table: Theater

id	theater	location	screens
92108	Theater 1	Mission Valley, San Diego, CA	5
91921	Theater 2	Chula Vista, San Diego, CA	7
92122	Theater 3	UTC, San Diego, CA	7

9.2.1.3 Table: Showtime

id	movieID	theaterID	administratorID	showDate	time	seatAvailability
00001	00001	92108	1	2026-08-15	2:00 PM	55
00002	00002	92108	1	2026-08-15	2:30 PM	55
00003	00002	91921	2	2026-08-15	3:00 PM	70
00004	00003	91921	2	2026-08-15	3:00 PM	75
00005	00001	92122	3	2026-08-15	1:00 PM	60
00006	00003	92122	3	2026-08-15	1:30 PM	60

9.2.1.4 Table: RegisteredCustomer

id	name	emailAddress	password	membershipID	preferredPaymentMethod
00001	Jack Sparrow	jsparrow1@gmail.com	E32Anjkc2Acjndeoo431alssmilakmm1Alkn2312asdes5a	M00001	PayPal
00002	John Wick	jwick2@gmail.com	9fK2mPq81xVhL7aNw0YcR4tBz6QeJuD3HsMiXn5Gp	M00002	Credit Card
00003	Eve Macarro	emacarro@gmail.com	b7RkL91vQx3NmP8cYf2ZaHt6UwE0JsG5DdAnX4Ve	M00003	Credit Card

9.2.1.5 Table: Administrator

id	name	emailAddress	password	accessLevel
1	Edward Teague	eteague@sdtheater.com	7f3c9e1a84d2b5c6f91e0a7d3b8c5e2f6d4a1b9c8e7f2d3a5c6b1e9f4d8a7c2	A0
2	Winston Scott	wscott@sdtheater.com	c9d1e7a4f2b8c3d6a5e9f0b1c7d2e4a8f6b3c1d9e5a7f2b4c8d6e1f9a3b5c7d	A0
3	Arthur Morgan	amorgan@sdtheater.com	1a9f4c7d2b8e5f3a6d1c9b7e4a2f8c5d0e6b3a1f9d7c4e2b8a5f6d1c3e9b7a4	A0

9.2.1.6 Table: Ticket

id	customerID	transactionID	showtimeID	administratorID	seatNumber	ticketType	ticketPrice	nftIdentifier	ticketStatus
1	00001	00001	00003	NULL	H13	Regular	18.99	0x8F3A7C1D9E5B2A64C18F7D9E3A5B6C2D	Valid
2	00001	00001	00003	NULL	H12	Regular	18.99	0xA1E5C9D7B3F8A6C2D4E9B7F1C5A3D8E6	Valid
3	00002	00002	00002	NULL	J9	Student	15.99	0xC4D8E1A7F3B9C6D2E5A8F1B7C3D9E4A6	Valid
4	00003	00003	00003	1	H16	Regular	18.99	0xF7B2C8D1A5E9C3F6B4D1A8E5C2F7B9D3	Refunded
5	99991	00004	00003	NULL	G15	Regular	21.99	0xB3F8D1A6C9E2F7A4D5C1E8B9A3F6C2D7	Valid

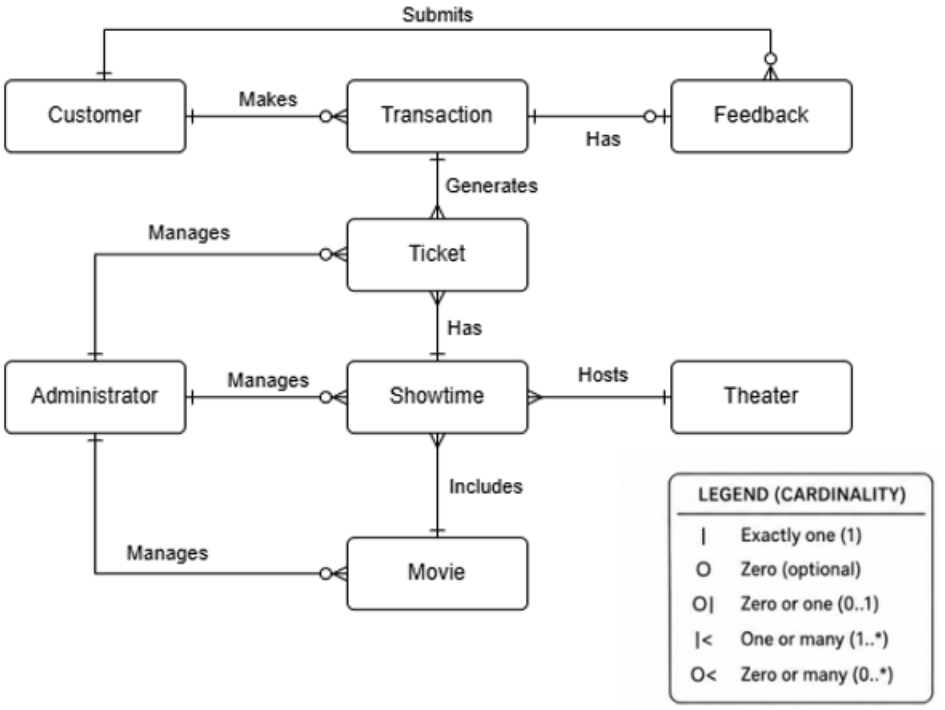
9.2.1.7 Table: Transaction

id	customerID	transactionDate	paymentMethod	totalAmount	transactionStatus
00001	00001	2026-08-10 1:35 PM	Credit Card	37.98	Completed
00002	00002	2026-08-10 2:05 PM	Credit Card	15.99	Completed
00003	00003	2026-08-10 2:12 PM	Credit Card	18.99	Refunded
00004	99991	2026-08-10 2:18 PM	Credit Card	21.99	Completed

9.2.1.8 Table: Feedback

id	customerID	transactionID	rating	comment	submissionDate
00001	00001	00001	4	“Process was quick and easy.”	2026-08-10 1:38 PM
00002	00002	00002	4	“Process was smooth and received my digital ticket immediately.”	2026-08-10 2:07 PM
00003	99991	00004	5	“Checkout process was simple and convenient.”	2026-08-10 2:21 PM

9.2.2 Entity Relationship (ER) Diagram



9.3 Database Security

Numerous security measures are utilized by the Theater Ticketing System in order to protect the private and personal information of customers. Access control for both customers and administrators is implemented to restrict database access from unqualified roles. Therefore, users are allowed to access and perform operations only authorized to their specific roles. For example, customers may only access their own personal account information including ticket purchase history as well as the ability to only purchase tickets. Administrators are not allowed to access the personal accounts of individual customers but rather can manage and modify system data such as available movies, showtimes, and tickets. These measures are intended to protect the privacy of users and prevent unauthorized parties from accessing unwanted information from the main database.

The system is also intended to enable protection of the individual personal and payment information of users. Rather than storing passwords in plain text, passwords are converted to a hashed format prior to entering the database. Specific payment attributes such as card numbers or security codes are never stored in the database. Instead, only surface-level information such as the preferred payment method from the customer is contained. These restrictions protect the private and personal data of users in the scenario where unauthorized access or a data breach does occur.

The database utilizes primary and secondary key relationships to connect entities to one another in order to confirm that the data stays accurate and consistent throughout. These relationships also prevent the duplication of data and invalid references so that existing data remains synchronized when updated.

These measures of security are enforced for the protection of user information as well as the reduction of data theft in the case that the database is jeopardized. Therefore, the precautions allow for a secure environment regarding the management of the system's data.

9.4 Backup and Recovery

The Theater Ticketing System is intended to implement a backup recovery plan to continue business and reduce the loss of data in the case of software/hardware failures, cyberattacks, or other potential setbacks.

- The main central database is to be automatically backed up in intervals of 24 hours.
- Smaller backups are to be performed hourly in order to preserve recent transactions and updates.
- Backup files are to be encrypted by industry-standard algorithms prior to their storage.
- These backups will be stored in distant locations from one another to minimize damage from physical disasters.
- Daily backups are to be saved for 30 days and monthly backups for one year prior to deletion.
- Recovery procedures should be available for the restoration of customer accounts, ticket transactions, movie schedules, and administrator logs.
- The target Recovery Time Objective will not exceed four hours after a major system failure.
- The target Recovery Point Objective will not exceed one hour of data loss.
- Backup recovery procedures are to be tested periodically in order to ensure successful data recovery in the event of a conflict.

9.5 Design Decisions and Tradeoffs

The Theater Ticketing System was intended to meet the functional requirements while balancing security, maintainability, and reliability measures.

Centralized Database: A main central database is utilized to contain all of the necessary data rather than separate databases for individual theater locations. This main database confirms that system components like customer information, seat availability, and tickets retain synchronization across all available theater locations. Unfortunately, the downside of this aspect is the concept that a single failure affecting the central database will affect the full system and therefore requires well-practiced backup and recovery procedures to reduce risk and negative effects.

Web-Based Architecture: The system was created as a web application in order for customers to purchase tickets manually from their personal devices or theater kiosks without any requirements of software installation. This allows for better user accessibility and reduces the necessity for software updates. Though, a stable network connection and constant availability of the server is required for this approach to function as intended.

Third-Party Payment Gateway: The system implements a certified third-party payment gateway to handle payment processing details. This approach reduces the need for complicated

development features and specific compliance requirements from the end of the ticketing system. Although, dependence on an external provider is required, and therefore unavoidable service interruptions are a possibility.

NFT-Based Digital Tickets: NFT-based tickets were selected as the preferred approach in order to provide unique ticket ownership and reduce possible cases of fraudulent ticket activity. Blockchain integration does increase complexity of development, though it ensures stronger security against the duplication and unapproved resale of tickets.

Multi-Factor Authentication: Administrator accounts contain a requirement of multi-factor authentication procedures to access the content within. Although multi-factor authentication very slightly increases user log-in time, it ensures security by significantly reducing risks of unauthorized access to important components of the system such as overall management and refunds.

Modular Architecture: The user interface, application logic, and payment service are fully separated into independent components of the overall system. Although this design was structurally more complicated to implement, it simplifies the addition of new features or future maintenance to the system.

9.6 Data Management Strategy

The Theater Ticketing System uses a centralized Database Management System (DBMS) to store and manage all operational data securely and efficiently.

The database shall maintain the following information:

- Customer account information and membership records.
- Administrator account information and audit logs.
- Movie information, showtimes, theater locations, and seat availability.
- Ticket purchases, reservations, refunds, and transaction history.
- NFT ticket identifiers and ownership information.
- Customer feedback and system-generated notifications.

To ensure data quality and consistency:

- Primary and foreign keys shall maintain relationships between customers, transactions, tickets, and showtimes.
- Database transactions shall follow ACID properties to ensure reliable ticket purchases and prevent duplicate seat assignments.
- Input validation shall be performed before data is stored.
- Sensitive customer information, including passwords and payment-related information, shall be encrypted or securely hashed.
- Access to database records shall follow role-based access control, limiting administrative privileges to authorized personnel.
- Audit logs shall record significant administrative actions, login attempts, refunds, and database modifications.

- Data retention policies shall comply with organizational policies and applicable privacy regulations.
- The database shall support concurrent access while maintaining real-time synchronization of seat availability across all theater locations.

These data management practices ensure the integrity, availability, confidentiality, and reliability of the Theater Ticketing System throughout its operation.

10. Design Specification Updates

The initial software design developed in Assignment 2 was reviewed and refined during the data management design process to better align the system architecture with the finalized requirements and database structure.

One significant modification was the addition of the **Theater** class to represent individual theater locations within the Theater Ticketing System. In the original design, theater information was stored as an attribute of the **Showtime** class. While this approach distinguished different theater locations, it did not accurately model the relationship between theaters and their associated showtimes. By introducing a dedicated **Theater** class, the updated design provides a clearer object-oriented representation of the system, improves data organization, reduces redundancy, and establishes a more maintainable relationship between theaters and showtimes.

Another enhancement was the addition of the **administratorID** attribute to the **Movie**, **Showtime**, and **Ticket** classes. This attribute records the identifier of the most recent administrator who modified each object, improving accountability, supporting audit tracking, and strengthening system security by providing a record of administrative actions.

The **AccountAuthentication** class was also updated to include a **logout()** function. This addition allows authenticated users to securely terminate their sessions, reducing the risk of unauthorized access and improving the overall security of the system.

The UML class diagram was updated to reflect these modifications and ensure consistency between the software design, functional requirements, and database architecture.

Aside from these enhancements, the overall software design developed in Assignment 2 continues to satisfy the system requirements. The remaining classes, relationships, and architectural components were retained because they remain consistent with the finalized functional requirements and the data management strategy.